

NeetCode 150

Not In 331 · Print Edition

32 questions · Python only · Bold black 6-up
Brute + Optimal · All 4 sites · Key insights · Quick Review

Contents

Sliding Window	M1
#567 Permutation in String (Medium)	
Stack	M1
#853 Car Fleet (Medium)	
Binary Search	M1
#875 Koko Eating Bananas (Medium)	
Linked List	M1
#138 Copy List with Random Pointer (Medium)	
Trees & BST	M1
#1448 Count Good Nodes in Binary Tree (Medium)	
Heap	E2 M1
#703 Kth Largest Element in a Stream (Easy)	
#1046 Last Stone Weight (Easy)	
#355 Design Twitter (Medium)	
Backtracking	M2
#40 Combination Sum II (Medium)	
#131 Palindrome Partitioning (Medium)	
Graphs	M1
#684 Redundant Connection (Medium)	
Graphs — Advanced	M2 H2
#743 Network Delay Time (Medium)	
#332 Reconstruct Itinerary (Hard)	
#1584 Min Cost to Connect All Points (Medium)	
#778 Swim in Rising Water (Hard)	

Dynamic Programming 1D	E1 M2
#746 Min Cost Climbing Stairs (Easy)	
#213 House Robber II (Medium)	
#647 Palindromic Substrings (Medium)	
Dynamic Programming 2D	M3
#518 Coin Change II (Medium)	
#494 Target Sum (Medium)	
#97 Interleaving String (Medium)	
Greedy	M5
#45 Jump Game II (Medium)	
#846 Hand of Straights (Medium)	
#1899 Merge Triplets to Form Target Triplet (Medium)	
#763 Partition Labels (Medium)	
#678 Valid Parenthesis String (Medium)	
Intervals (Greedy)	H1
#1851 Minimum Interval to Include Each Query (Hard)	
Math & Geometry	E2 M2
#202 Happy Number (Easy)	
#66 Plus One (Easy)	
#43 Multiply Strings (Medium)	
#2013 Detect Squares (Medium)	
Bit Manipulation	M1
#371 Sum of Two Integers (Medium)	

Sliding Window

NeetCode 150 · Not In 331 · 1 question

Sliding Window

#567 Permutation in String

MED

Time Complexity: $O(n)$ where n is the length of $s2$ (each character is processed a constant number of times). Space Complexity: $O(1)$ since the frequency count arrays have a fixed size (26 for lowercase English letters).

Key Insights

- Use a sliding window approach to examine substrings of $s2$ of length equal to $s1$.
- Maintain frequency counts of characters for both $s1$ and the current window in $s2$.
- Compare the frequency counts to determine if a valid permutation exists.
- Increment the window by adding a new character and removing the leftmost one, keeping the operation efficient.

Brute Force (Python)

```
# Brute Force  $O(n \cdot k)$  - check every window of size len(s1)
from collections import Counter
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        k = len(s1)
        need = Counter(s1)
        for i in range(len(s2) - k + 1):
            if Counter(s2[i:i + k]) == need:
                return True
        return False
```

Optimal (Python)

```
# Optimal  $O(n)$  - fixed sliding window with 26-element arrays
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        if len(s1) > len(s2): return False
        need = [0] * 26
        have = [0] * 26
        for c in s1:
            need[ord(c) - ord('a')] += 1
        k = len(s1)
        for i, c in enumerate(s2):
            have[ord(c) - ord('a')] += 1
            if i >= k:
                have[ord(s2[i - k]) - ord('a')] -= 1
            if i >= k - 1 and have == need:
                return True
        return False
```

Best Answers — All Sites (Python)

• WalkCC

```
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        count = collections.Counter(s1)
        required = len(s1)

        for r, c in enumerate(s2):
            count[c] -= 1
            if count[c] >= 0:
                required -= 1
            if r >= len(s1): # The window is oversized.
                count[s2[r - len(s1)]] += 1
                if count[s2[r - len(s1)]] > 0:
                    required += 1
            if required == 0:
                return True

        return False
```

• LeetCode

p.7

```
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        cnt = Counter(s1)
        need = len(cnt)
        m = len(s1)
        for i, c in enumerate(s2):
            cnt[c] -= 1
            if cnt[c] == 0:
                need -= 1
            if i >= m:
                cnt[s2[i - m]] += 1
                if cnt[s2[i - m]] == 1:
                    need += 1
            if need == 0:
                return True
        return False
```

• SimplyLeet

p.8

```
# Python solution with line-by-line comments

def checkInclusion(s1: str, s2: str) -> bool:
    # Get lengths of the input strings
    len1, len2 = len(s1), len(s2)
    # Edge case: s1 cannot be a substring if longer than s2
    if len1 > len2:
        return False

    # Initialize frequency arrays for s1 and for the first window of s2
    s1_count = [0] * 26
    window_count = [0] * 26

    # Build frequency counts for s1 and the initial window in s2
    for i in range(len1):
        s1_count[ord(s1[i]) - ord('a')] += 1
        window_count[ord(s2[i]) - ord('a')] += 1

    # Check if initial window is a permutation of s1
    if s1_count == window_count:
        return True
```

p.9

```
# Slide the window across s2, updating the counts on the fly
for i in range(len1, len2):
    # Add the new character from the right end of the window
    window_count[ord(s2[i]) - ord('a')] += 1
    # Remove the character that is no longer in the window
    window_count[ord(s2[i - len1]) - ord('a')] -= 1

    # Check if current window matches s1's frequency distribution
    if window_count == s1_count:
        return True

    return False
```

```
# Example usage:
# print(checkInclusion("ab", "eidbaooo")) # Expected output: True
```

• LC.ca

p.10

```
class Solution:
    def checkInclusion(self, s1: str, s2: str) -> bool:
        n, m = len(s1), len(s2)
        if n > m:
            return False
        cnt = Counter()
        for a, b in zip(s1, s2):
            cnt[a] -= 1
            cnt[b] += 1
        diff = sum(x != 0 for x in cnt.values())
        if diff == 0:
            return True
        for i in range(n, m):
            a, b = s2[i - n], s2[i]

            if cnt[b] == 0:
                diff += 1
            cnt[b] += 1
            if cnt[b] == 0:
                diff -= 1

            if cnt[a] == 0:
                diff += 1
            cnt[a] -= 1
            if cnt[a] == 0:
                diff -= 1

            if diff == 0:
                return True
        return False
```

p.11

Stack

NeetCode 150 · Not In 331 · 1 question

#853 Car Fleet

MED

Time Complexity: $O(n \log n)$ due to the sorting step. Space Complexity: $O(n)$ for storing the computed times and sorted car information.

Key Insights

- Since cars cannot pass each other, the relative order (by starting position) is maintained.
- Sorting the cars based on their starting positions (descending order) helps to simulate the catching-up process.
- Computing the time each car takes to reach the target is critical.
- Compare the current car's time to the time of the fleet ahead: if it takes longer, it forms a new fleet; otherwise, it gets merged.
- A stack-like approach (tracking the slowest arrival time seen so far) reduces the problem to a single pass after sorting.

Brute Force (Python)

```
# Brute Force  $O(n^2)$  - simulate merging for every pair
class Solution:
    def carFleet(self, target: int, position: list, speed: list) -> int:
        pairs = sorted(zip(position, speed), reverse=True)
        fleets = 0
        times = [(target - p) / s for p, s in pairs]
        i = 0
        while i < len(times):
            fleets += 1
            t = times[i]; i += 1
            while i < len(times) and times[i] <= t:
                i += 1
        return fleets
```

p.13

```
# Python implementation of Car Fleet
def carFleet(target, position, speed):
    # Pair up position and speed and sort by starting position in descending order.
    cars = sorted(zip(position, speed), key=lambda car: car[0], reverse=True)

    fleets = 0 # Count of car fleets
    last_time = 0.0 # Time taken by the last fleet to reach the target

    # Process each car from closest to farthest from the target.
    for pos, spd in cars:
        # Calculate time for this car to reach the target
        time = (target - pos) / spd

        # If this car takes longer than the last fleet time, it starts a new fleet.
        if time > last_time:
            fleets += 1
            last_time = time # Update the last fleet time with this car's time

    return fleets

# Example usage:
# target = 12, position = [10,8,0,5,3], speed = [2,4,1,1,3]
# print(carFleet(12, [10,8,0,5,3], [2,4,1,1,3])) # Output: 3
```

• LC.ca

p.16

Optimal (Python)

```
# Optimal  $O(n \log n)$  - monotonic stack of arrival times
class Solution:
    def carFleet(self, target: int, position: list, speed: list) -> int:
        pairs = sorted(zip(position, speed), reverse=True)
        stack = []
        for pos, spd in pairs:
            t = (target - pos) / spd
            if not stack or t > stack[-1]:
                stack.append(t)
        return len(stack)
```

Best Answers — All Sites (Python)

• WalkCC

```
class Solution:
    def carFleet(self, target: int, position: list[int], speed: list[int]) -> int:
        ans = 0
        times = [
            float(target - p) / s for p, s in sorted(zip(position, speed), reverse=True)]

        maxTime = 0 # the time of the slowest car to reach the target

        for time in times:
            # A car needs more time to reach the target, so it becomes the slowest.
            if time > maxTime:
                maxTime = time
                ans += 1

        return ans
```

p.14

```
class Solution:
    def carFleet(self, target: int, position: List[int], speed: List[int]) -> int:
        idx = sorted(range(len(position)), key=lambda i: position[i])
        ans = pre = 0
        for i in idx[::-1]:
            t = (target - position[i]) / speed[i]
            if t > pre:
                ans += 1
                pre = t
        return ans
```

p.17

• LeetDoocs

```
class Solution:
    def carFleet(self, target: int, position: List[int], speed: List[int]) -> int:
        idx = sorted(range(len(position)), key=lambda i: position[i])
        ans = pre = 0
        for i in idx[::-1]:
            t = (target - position[i]) / speed[i]
            if t > pre:
                ans += 1
                pre = t
        return ans
```

• SimplyLeet

p.15

Binary Search

NeetCode 150 · Not In 331 · 1 question

p.18

#875 Koko Eating Bananas

MED

Time Complexity: $O(n \log(\max(\text{pile})))$ where n is the number of piles and $\max(\text{pile})$ is the maximum bananas in a pile. Space Complexity: $O(1)$ extra space beyond the input.

Key Insights

- The required eating speed k lies in the range $[1, \max(\text{piles})]$.
- For a given k , one can calculate the total hours needed by summing up $\text{ceil}(\text{pile} / k)$ for each pile.
- The problem can be solved efficiently using binary search to find the minimum k that satisfies the time constraint.
- Early termination in the hour calculation can be used if the computed hours exceed h .

Brute Force (Python)

```
# Brute Force  $O(n \cdot m)$  – try every  $k$  from 1 to  $\max(\text{piles})$ 
import math
class Solution:
    def minEatingSpeed(self, piles: list, h: int) -> int:
        for k in range(1, max(piles) + 1):
            if sum(math.ceil(p / k) for p in piles) <= h:
                return k
```

Optimal (Python)

p.19

```
# Optimal  $O(n \log m)$  – binary search on  $k$ 
import math
class Solution:
    def minEatingSpeed(self, piles: list, h: int) -> int:
        lo, hi = 1, max(piles)
        while lo < hi:
            mid = (lo + hi) // 2
            if sum(math.ceil(p / mid) for p in piles) <= h:
                hi = mid
            else:
                lo = mid + 1
        return lo
```

Best Answers — All Sites (Python)

• WalkCC

```
class Solution:
    def minEatingSpeed(self, piles: list[int], h: int) -> int:
        def eatHours(m: int) -> bool:
            """Returns the hours to eat all the piles with speed m."""
            return sum((pile - 1) // m + 1 for pile in piles)
        l = 1
        r = max(piles)
        return bisect.bisect_left(range(l, r), True,
                                key=lambda m: eatHours(m) <= h) + 1
```

• LeetCode

```
class Solution:
    def minEatingSpeed(self, piles: List[int], h: int) -> int:
        def check(k: int) -> bool:
            return sum((x + k - 1) // k for x in piles) <= h

        return 1 + bisect_left(range(1, max(piles) + 1), True,
                              key=check)
```

p.20

• SimplyLeet

```
def minEatingSpeed(piles, h):
    # Helper function to determine if Koko can finish with speed k
    def canFinish(k):
        hours = 0
        for pile in piles:
            # Calculate hours by using ceiling division for each pile
            hours += (pile + k - 1) // k
        return hours <= h

    left, right = 1, max(piles) # Speed is between 1 and the max pile
    element
    while left < right:
        mid = (left + right) // 2 # Candidate eating speed
        if canFinish(mid):
            right = mid # Try finding a smaller valid speed
        else:
            left = mid + 1 # Increase the speed because mid is too slow
    return left

# Example usage:
piles = [3, 6, 7, 11]
h = 8
print(minEatingSpeed(piles, h)) # Expected output: 4
```

• LC.ca

p.21

```
class Solution:
    def minEatingSpeed(self, piles: List[int], h: int) -> int:
        left, right = 1, int(1e9)
        while left < right:
            mid = (left + right) >> 1
            s = sum((x + mid - 1) // mid for x in piles)
            if s <= h:
                right = mid
            else:
                left = mid + 1
        return left
```

p.22

Linked List

NeetCode 150 · Not In 331 · 1 question

#138 Copy List with Random Pointer

MED

Time Complexity: $O(n)$ – Each node is processed a constant number of times. Space Complexity: $O(n)$ – A hash map is used to store the mapping from original nodes to their copies.

Key Insights

- Use a hash map (or dictionary) to map each original node to its corresponding copy.
- Perform two passes: the first to create a copy of each node, and the second to assign the next and random pointers using the hash map.
- An alternative interleaving approach exists but using a hash table is typically more straightforward.

Brute Force (Python)

```
# Brute Force  $O(n)$  space – two-pass hashmap
class Solution:
    def copyRandomList(self, head):
        if not head: return None
        mapping = {}
        curr = head
        while curr:
            mapping[curr] = Node(curr.val)
            curr = curr.next
        curr = head
        while curr:
            if curr.next: mapping[curr].next = mapping[curr.next]
            if curr.random: mapping[curr].random = mapping[curr.random]
            curr = curr.next
        return mapping[head]
```

Optimal (Python)

p.24

p.23

```
# Optimal O(1) space - interleave clones, wire randoms, split
class Solution:
    def copyRandomList(self, head):
        if not head: return None
        curr = head
        while curr:
            clone = Node(curr.val)
            clone.next = curr.next
            curr.next = clone
            curr = clone.next
        curr = head
        while curr:
            if curr.random:
                curr.next.random = curr.random.next
            curr = curr.next.next
        dummy = Node(0)
        clone_curr = dummy
        curr = head
        while curr:
            clone_curr.next = curr.next
            curr.next = curr.next.next
            clone_curr = clone_curr.next

        curr = curr.next
        return dummy.next
```

Best Answers — All Sites (Python)

• WalkCC

p.25

```
class Solution:
    def copyRandomList(self, head: 'Node') -> 'Node':
        if not head:
            return None
        if head in self.map:
            return self.map[head]

        newNode = Node(head.val)
        self.map[head] = newNode
        newNode.next = self.copyRandomList(head.next)
        newNode.random = self.copyRandomList(head.random)
        return newNode

        map = {}
```

• LeetCode

p.26

```
"""
# Definition for a Node.
class Node:
    def __init__(self, x: int, next: 'Node' = None, random: 'Node' = None):
        self.val = int(x)
        self.next = next
        self.random = random
"""

class Solution:
    def copyRandomList(self, head: "Optional[Node]") -> "Optional[Node]":
        d = {}
        dummy = tail = Node(0)
        cur = head
        while cur:
            node = Node(cur.val)
            tail.next = node
            tail = tail.next
            d[cur] = node
            cur = cur.next
        cur = head

        while cur:
            d[cur].random = d[cur.random] if cur.random else None
            cur = cur.next
        return dummy.next
```

• SimplyLeet

p.27

```
# Definition for a Node.
class Node:
    def __init__(self, val, next=None, random=None):
        self.val = val # value of the node.
        self.next = next # pointer to the next node.
        self.random = random # pointer to a random node.

class Solution:
    def copyRandomList(self, head: 'Node') -> 'Node':
        # Return None if the head is None.
        if not head:
            return None

        # Dictionary to hold old node as key and new node as value.
        old_to_new = {}

        # First pass: create a copy of each node.
        current = head
        while current:
            old_to_new[current] = Node(current.val)
            current = current.next

        # Second pass: assign next and random pointers for the copied nodes.
        current = head
        while current:
            # Using dict.get ensures proper assignment or returns None.
            old_to_new[current].next = old_to_new.get(current.next)
            old_to_new[current].random = old_to_new.get(current.random)
            current = current.next

        # Return the head of the new copied linked list.
        return old_to_new[head]
```

• LC.ca

p.28

```
# Definition for singly-linked list with a random pointer.
# class RandomListNode(object):
#     def __init__(self, x):
#         self.label = x
#         self.next = None
#         self.random = None

class Solution:
    def copyRandomList(self, head: "Node") -> "Node":
        if head is None:
            return None
        cur = head
        while cur: # copy nodes
            node = Node(cur.val, cur.next)
            cur.next = node
            cur = node.next

        cur = head
        while cur: # copy random pointers
            if cur.random:
                cur.next.random = cur.random.next
            cur = cur.next.next
```

p.29

```
ans = head.next
cur = head
while cur: # cut into 2 lists
    nxt = cur.next
    if nxt:
        cur.next = nxt.next
    cur = nxt
return ans

class Solution: # with a map
    def copyRandomList(self, head: 'Node') -> 'Node':
        if not head:
            return None

        node_map = {}

        # Create the copy nodes without next and random connections
        current = head
        while current:
            node_map[current] = Node(current.val)
            current = current.next

        # Assign next and random connections for the copy nodes
        current = head
        while current:
            copy_node = node_map[current]
            copy_node.next = node_map.get(current.next)
            copy_node.random = node_map.get(current.random)
            current = current.next

        return node_map[head]
```

p.30

Trees & BST

NeetCode 150 · Not In 331 · 1 question

Trees & BST

#1448 Count Good Nodes in Binary Tree

MED

Time Complexity: $O(n)$, where n is the number of nodes, since each node is visited exactly once. Space Complexity: $O(h)$, where h is the height of the tree ($O(n)$ in the worst-case of a skewed tree).

Key Insights

- Perform a tree traversal (DFS or BFS) from the root.
- Keep track of the maximum value encountered along the current path.
- A node is "good" if its value is greater than or equal to the maximum value seen so far.
- Update the maximum value as you traverse deeper in the tree.
- The root is always a good node.

Brute Force (Python)

```
# DFS - track running max on each root-to-node path
class Solution:
    def goodNodes(self, root) -> int:
        def dfs(node, max_val):
            if not node: return 0
            good = 1 if node.val >= max_val else 0
            new_max = max(max_val, node.val)
            return good + dfs(node.left, new_max) + dfs(node.right,
new_max)
        return dfs(root, float('-inf'))
```

Optimal (Python)

```
# Optimal  $O(n)$  - iterative DFS with explicit stack
from collections import deque
class Solution:
    def goodNodes(self, root) -> int:
        count = 0
        stack = [(root, float('-inf'))]
        while stack:
            node, max_val = stack.pop()
            if not node: continue
            if node.val >= max_val:
                count += 1
            new_max = max(max_val, node.val)
            stack.append((node.left, new_max))
            stack.append((node.right, new_max))
        return count
```

Best Answers — All Sites (Python)

• [LeetDoocs](#)

p.31

p.32

p.33

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def goodNodes(self, root: TreeNode) -> int:
        def dfs(root: TreeNode, mx: int):
            if root is None:
                return
            nonlocal ans
            if mx <= root.val:
                ans += 1
                mx = root.val
            dfs(root.left, mx)
            dfs(root.right, mx)

        ans = 0
        dfs(root, -1000000)
        return ans
```

• [SimplyLeet](#)

p.34

```
# Definition for a binary tree node.
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    def goodNodes(self, root: TreeNode) -> int:
        # Helper function to perform DFS traversal.
        def dfs(node, currentMax):
            # If we reach a null node, there are no further nodes to
count.

            if not node:
                return 0

            # Check if current node is good.
            count = 1 if node.val >= currentMax else 0
            # Update the current maximum value.
            newMax = max(currentMax, node.val)

            # Recursively count good nodes in the left and right
subtrees.

            count += dfs(node.left, newMax)

            count += dfs(node.right, newMax)
            return count

        # Start DFS traversal from root with initial maximum value as
root's value.
        return dfs(root, root.val)
```

• [LC.ca](#)

Sheet 6 / 41 · LeetMastery NeetCode 32 · Not In 331

p.35

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def goodNodes(self, root: TreeNode) -> int:
        def dfs(root: TreeNode, mx: int):
            if root is None:
                return
            nonlocal ans
            if mx <= root.val:
                ans += 1
                mx = root.val
            dfs(root.left, mx)
            dfs(root.right, mx)

        ans = 0
        dfs(root, -1000000)
        return ans
```

p.36

Heap

NeetCode 150 · Not In 331 · 3 questions

Heap

#703 Kth Largest Element in a Stream

EAS

Time Complexity: $O(\log k)$ per add operation, $O(n \log k)$ to build the initial heap. Space Complexity: $O(k)$ for maintaining the heap.

Key Insights

- Use a min-heap (priority queue) of fixed size k .
- The heap always contains the k largest elements; the smallest element in the heap is the k th largest overall.
- When a new test score is added:
- If the heap has less than k elements, simply add the score.
- If the heap is full and the new score is greater than the smallest (heap root), replace the root.
- Each add operation runs in $O(\log k)$ time.

Brute Force (Python)

```
# Brute Force  $O(n \log n)$  per add – sort entire list each time
class KthLargest:
    def __init__(self, k: int, nums: list):
        self.k = k; self.data = nums[:]
    def add(self, val: int) -> int:
        self.data.append(val)
        self.data.sort(reverse=True)
        return self.data[self.k - 1]
```

Optimal (Python)

```
# Optimal  $O(\log k)$  per add – fixed-size min-heap
import heapq
class KthLargest:
    def __init__(self, k: int, nums: list):
        self.k = k
        self.heap = nums[:]
        heapq.heapify(self.heap)
        while len(self.heap) > k:
            heapq.heappop(self.heap)
    def add(self, val: int) -> int:
        heapq.heappush(self.heap, val)
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)
        return self.heap[0]
```

Best Answers — All Sites (Python)

- [LeetCode](#)

p.37

p.38

p.39

```
class KthLargest:
    def __init__(self, k: int, nums: List[int]):
        self.k = k
        self.min_q = []
        for x in nums:
            self.add(x)
    def add(self, val: int) -> int:
        heappush(self.min_q, val)
        if len(self.min_q) > self.k:
            heappop(self.min_q)
        return self.min_q[0]
```

Your KthLargest object will be instantiated and called as such:
obj = KthLargest(k, nums)
param_1 = obj.add(val)

- [SimplyLeet](#)

```
import heapq
class KthLargest:
    def __init__(self, k, nums):
        # Save kth value and initialize the min-heap.
        self.k = k
        self.min_heap = nums
        heapq.heapify(self.min_heap)
        # Reduce the heap size to exactly k elements.
        while len(self.min_heap) > k:
            heapq.heappop(self.min_heap)
    def add(self, val):
        if len(self.min_heap) < self.k:
            heapq.heappush(self.min_heap, val)
        elif val > self.min_heap[0]:
            heapq.heapreplace(self.min_heap, val)
        # The kth largest element is at the heap root.
        return self.min_heap[0]
```

Example usage:
kthLargest = KthLargest(3, [4, 5, 8, 2])

```
# print(kthLargest.add(3)) # returns 4
# print(kthLargest.add(5)) # returns 5
# print(kthLargest.add(10)) # returns 5
# print(kthLargest.add(9)) # returns 8
# print(kthLargest.add(4)) # returns 8
```

- [LC.ca](#)

```
class KthLargest:
    def __init__(self, k: int, nums: List[int]):
        self.q = []
        self.size = k
        for num in nums:
            self.add(num)
    def add(self, val: int) -> int:
        heappush(self.q, val)
        if len(self.q) > self.size:
            heappop(self.q)
        return self.q[0]
```

Your KthLargest object will be instantiated and called as such:
obj = KthLargest(k, nums)
param_1 = obj.add(val)

p.40

p.41

p.42

Heap

#1046 Last Stone Weight

EAS

Time Complexity: $O(n \log n)$ - Each stone removal and insertion takes $O(\log n)$ and in the worst-case scenario, we perform $O(n)$ operations. Space Complexity: $O(n)$ - The heap stores all the stones initially.

Key Insights

- Use a max-heap (or simulate one) to always access the two largest stones quickly.
- If the two stones are equal, both are removed; otherwise, the difference is pushed back into the heap.
- Continue the process until the heap has one or no stone left.
- For languages without a max-heap in their standard library (e.g., Python), simulate one by pushing negative values.

Brute Force (Python)

```
# Brute Force  $O(n^2)$  - sort array on each iteration
class Solution:
    def lastStoneWeight(self, stones: List) -> int:
        stones = stones[:]
        while len(stones) > 1:
            stones.sort()
            y, x = stones.pop(), stones.pop()
            if y != x: stones.append(y - x)
        return stones[0] if stones else 0
```

Optimal (Python)

p.43

```
# Optimal  $O(n \log n)$  - max-heap via negation
import heapq
class Solution:
    def lastStoneWeight(self, stones: List) -> int:
        heap = [-s for s in stones]
        heapq.heapify(heap)
        while len(heap) > 1:
            y = -heapq.heappop(heap)
            x = -heapq.heappop(heap)
            if y != x: heapq.heappush(heap, -(y - x))
        return -heap[0] if heap else 0
```

Best Answers — All Sites (Python)

• WalkCC

```
class Solution:
    def lastStoneWeight(self, stones: List[int]) -> int:
        pq = [-stone for stone in stones]
        heapq.heapify(pq)

        while len(pq) >= 2:
            n1 = -heapq.heappop(pq)
            n2 = -heapq.heappop(pq)
            if n1 != n2:
                heapq.heappush(pq, -(n1 - n2))

        return 0 if not pq else -pq[0]
```

• LeetCode

p.44

```
class Solution:
    def lastStoneWeight(self, stones: List[int]) -> int:
        h = [-x for x in stones]
        heapq.heapify(h)
        while len(h) > 1:
            y, x = -heappop(h), -heappop(h)
            if x != y:
                heappush(h, x - y)
        return 0 if not h else -h[0]
```

• SimplyLeet

```
import heapq

def lastStoneWeight(stones):
    # Create a max heap by inserting negative values of stones
    max_heap = [-stone for stone in stones]
    heapq.heapify(max_heap)

    # While more than one stone remains
    while len(max_heap) > 1:
        # Get the heaviest stone (largest value, but stored as negative)
        heaviest = -heapq.heappop(max_heap)
        # Get the next heaviest stone
        next_heaviest = -heapq.heappop(max_heap)

        # If the stones are not equal, push the difference back into the heap
        if heaviest != next_heaviest:
            diff = heaviest - next_heaviest
            heapq.heappush(max_heap, -diff)

    # If there is a stone left, return its weight. Otherwise, return 0.
    return -max_heap[0] if max_heap else 0
```

p.45

```
# Example usage:
# print(lastStoneWeight([2,7,4,1,8,1]))
```

• LC.ca

```
class Solution:
    def lastStoneWeight(self, stones: List[int]) -> int:
        h = [-x for x in stones]
        heapq.heapify(h)
        while len(h) > 1:
            y, x = -heappop(h), -heappop(h)
            if x != y:
                heappush(h, x - y)
        return 0 if not h else -h[0]
```

p.46

Heap

#355 Design Twitter

MED

Time Complexity: - postTweet: $O(1)$ - follow / unfollow: $O(1)$ - getNewsFeed: $O(N \log K)$ where N is the number of followees and K is the average number of tweets considered (capped at 10 tweets per merge iteration) Space Complexity: - $O(U + T)$ where U is the number of users (for follow mappings) and T is the total number of tweets stored.

Key Insights

- Use a global timestamp to order tweets as they are posted.
- Maintain a mapping of each user to the list of tweets they posted.
- Maintain a mapping of each user to the set of users they follow.
- For getNewsFeed, merge tweets from the user and their followees using a heap to efficiently retrieve the 10 most recent tweets.
- Ensure that a user always follows themselves to simplify retrieving their own tweets.

Brute Force (Python)

```
# Brute Force - collect all tweets, sort, return top 10
from collections import defaultdict
class Twitter:
    def __init__(self):
        self.tweets = defaultdict(List); self.follows = defaultdict(set); self.time = 0
    def postTweet(self, userId, tweetId):
        self.tweets[userId].append((self.time, tweetId)); self.time += 1
    def getNewsFeed(self, userId):
        self.follows[userId].add(userId)
        feed = []
        for fid in self.follows[userId]: feed.extend(self.tweets[fid])
        feed.sort(reverse=True)
        return [tid for _, tid in feed[:10]]
    def follow(self, f, fd): self.follows[f].add(fd)
    def unfollow(self, f, fd): self.follows[f].discard(fd)
```

Optimal (Python)

p.47

p.48

```
# Optimal - heap merge over followees' most-recent tweets
import heapq
from collections import defaultdict
class Twitter:
    def __init__(self):
        self.tweets = defaultdict(list); self.follows = defaultdict(set); self.time = 0
    def postTweet(self, userId, tweetId):
        self.tweets[userId].append((self.time, tweetId)); self.time += 1
    def getNewsFeed(self, userId):
        self.follows[userId].add(userId)
        heap, res = [], []
        for fid in self.follows[userId]:
            tw = self.tweets[fid]
            if tw:
                idx = len(tw) - 1
                t, tid = tw[idx]
                heapq.heappush(heap, (-t, tid, fid, idx))
        while heap and len(res) < 10:
            neg_t, tid, fid, idx = heapq.heappop(heap)
            res.append(tid)
            if idx > 0:
                idx -= 1
                t, tid2 = self.tweets[fid][idx]
                heapq.heappush(heap, (-t, tid2, fid, idx))
        return res
    def follow(self, f, fd): self.follows[f].add(fd)
    def unfollow(self, f, fd): self.follows[f].discard(fd)
```

Best Answers — All Sites (Python)

• WalkCC

p.49

```
class Twitter:
    def __init__(self):
        self.timer = itertools.count(step=-1)
        self.tweets = collections.defaultdict(deque)
        self.followees = collections.defaultdict(set)

    def postTweet(self, userId: int, tweetId: int) -> None:
        self.tweets[userId].appendleft((next(self.timer), tweetId))
        if len(self.tweets[userId]) > 10:
            self.tweets[userId].pop()

    def getNewsFeed(self, userId: int) -> list[int]:
        tweets = list(
            heapq.merge(
                *
                (self.tweets[followee]
                 for followee in self.followees[userId] | {userId}))
        )
        return [tweetId for _, tweetId in tweets[:10]]

    def follow(self, followerId: int, followeeId: int) -> None:
        self.followees[followerId].add(followeeId)

    def unfollow(self, followerId: int, followeeId: int) -> None:
        self.followees[followerId].discard(followeeId)
```

• LeetCode

p.50

```
class Twitter:
    def __init__(self):
        """
        Initialize your data structure here.
        """
        self.user_tweets = defaultdict(list)
        self.user_following = defaultdict(set)
        self.tweets = defaultdict()
        self.time = 0

    def postTweet(self, userId: int, tweetId: int) -> None:
        """
        Compose a new tweet.
        """
        self.time += 1
        self.user_tweets[userId].append(tweetId)
        self.tweets[tweetId] = self.time

    def getNewsFeed(self, userId: int) -> List[int]:
        """
        Retrieve the 10 most recent tweet ids in the user's news feed.
        Each item in the news feed must be posted by users who the user followed
        or by the user herself. Tweets must be ordered from most recent to least
        recent.
        """
```

p.51

```
following = self.user_following[userId]
users = set(following)
users.add(userId)
tweets = [self.user_tweets[u][::-1][:10] for u in users]
tweets = sum(tweets, [])
return nlargest(10, tweets, key=lambda tweet:
self.tweets[tweet])

def follow(self, followerId: int, followeeId: int) -> None:
    """
    Follower follows a followee. If the operation is invalid, it
    should be a no-op.
    """
    self.user_following[followerId].add(followeeId)

def unfollow(self, followerId: int, followeeId: int) -> None:
    """
    Follower unfollows a followee. If the operation is invalid, it
    should be a no-op.
    """
    following = self.user_following[followerId]
    if followeeId in following:
        following.remove(followeeId)

# Your Twitter object will be instantiated and called as such:
# obj = Twitter()
# obj.postTweet(userId,tweetId)
# param_2 = obj.getNewsFeed(userId)
# obj.follow(followerId,followeeId)
# obj.unfollow(followerId,followeeId)
```

• SimplyLeet

p.52

```
# Python implementation of the Twitter design problem
import heapq

class Twitter:
    def __init__(self):
        # Mapping from userId to a set of followeeIds
        self.follows = {}
        # Mapping from userId to list of (timestamp, tweetId)
        self.tweets = {}
        # Global timestamp to order tweets
        self.timestamp = 0

    def postTweet(self, userId: int, tweetId: int) -> None:
        # Ensure the user follows itself for easier news feed retrieval
        if userId not in self.follows:
            self.follows[userId] = set()
        self.follows[userId].add(userId)

        # Append the tweet with the current timestamp, then increment
        timestamp
        if userId not in self.tweets:
            self.tweets[userId] = []
        self.tweets[userId].append((self.timestamp, tweetId))
```

p.53

```
self.timestamp += 1

def getNewsFeed(self, userId: int) -> list:
    # If the user does not exist in follows mapping, return empty
    feed

    if userId not in self.follows:
        return []

    # Use a heap (as a max-heap using negative timestamp)
    heap = []
    # Iterate over all followees of the user
    for followeeId in self.follows[userId]:
        if followeeId in self.tweets and self.tweets[followeeId]:
            # Start from the most recent tweet of the followee
            index = len(self.tweets[followeeId]) - 1
            ts, tid = self.tweets[followeeId][index]
            # Push the tweet along with followeeId and the index of
            the tweet in the list
            heapq.heappush(heap, (-ts, tid, followeeId, index))

    result = []
    # Retrieve up to 10 tweets
    while heap and len(result) < 10:
        neg_ts, tweetId, followeeId, idx = heapq.heappop(heap)
```

p.54

```

        result.append(tweetId)
        # If there is an older tweet from this followee, push it
        onto the heap
        if idx - 1 >= 0:
            ts, tid = self.tweets[followeeId][idx - 1]
            heapq.heappush(heap, (-ts, tid, followeeId, idx - 1))
        return result

def follow(self, followerId: int, followeeId: int) -> None:
    # A user should always follow itself
    if followerId not in self.follows:
        self.follows[followerId] = set()
    self.follows[followerId].add(followerId)
    # Add followeeId to follower's follow set if not self-follow
    if followerId != followeeId:
        self.follows[followerId].add(followeeId)

def unfollow(self, followerId: int, followeeId: int) -> None:
    # A user cannot unfollow itself
    if followerId in self.follows and followeeId != followerId:
        self.follows[followerId].discard(followeeId)

```

• LC.ca

p.55

```

'''
heapq.nlargest(n, iterable, key=None) is used to
find the n largest elements from a dataset defined by iterable.

people = [
    {"name": "Alice", "age": 30},
    {"name": "Bob", "age": 25},
    {"name": "Carol", "age": 40},
    {"name": "Dave", "age": 20}
]

# Using nlargest to find the two oldest people
oldest_two = heapq.nlargest(2, people, key=lambda person: person["age"])

print(oldest_two)
# Output: [{'name': 'Carol', 'age': 40}, {'name': 'Alice', 'age': 30}]

'''

from collections import defaultdict
from heapq import nlargest

```

p.56

```

from typing import List

class Twitter:
    def __init__(self):
        '''
        Initialize your data structure here.
        '''
        self.user_tweets = defaultdict(list)
        self.user_following = defaultdict(set)
        self.tweet_time = defaultdict(int) # id => timestamp
        self.time = 0

    def postTweet(self, userId: int, tweetId: int) -> None:
        '''
        Compose a new tweet.
        '''
        self.time += 1
        self.user_tweets[userId].append(tweetId)
        self.tweet_time[tweetId] = self.time

    def getNewsFeed(self, userId: int) -> List[int]:
        '''

```

p.57

```

        Retrieve the 10 most recent tweet ids in the user's news feed.
        Each item in the news feed must be posted by users who the user followed
        or by the user herself. Tweets must be ordered from most recent to least
        recent.
        '''
        following = self.user_following[userId]
        users = set(following)
        users.add(userId) # should see my own tweets too
        tweets = [self.user_tweets[u][::-1][:10] for u in users] # Get
        the 10 most recent tweets
        '''
        unconventional way to flatten a list of lists into a single
        list:
        lists = [[1, 2, 3], [4, 5], [6]]
        flattened = sum(lists, [])
        print(flattened) # Output: [1, 2, 3, 4, 5, 6]

        or, another way getting flattened tweets:
        flattened_tweets = [tweet for each_user_tweets in tweets
        for tweet in each_user_tweets]
        '''
        tweets = sum(tweets, [])
        return nlargest(10, tweets, key=lambda tweet:
        self.tweet_time[tweet])

    def follow(self, followerId: int, followeeId: int) -> None:
        '''
        Follower follows a followee. If the operation is invalid, it
        should be a no-op.
        '''

```

p.58

```

        self.user_following[followerId].add(followeeId)

    def unfollow(self, followerId: int, followeeId: int) -> None:
        '''
        Follower unfollows a followee. If the operation is invalid, it
        should be a no-op.
        '''
        following = self.user_following[followerId]
        if followeeId in following:
            following.remove(followeeId)

        # Your Twitter object will be instantiated and called as such:
        # obj = Twitter()
        # obj.postTweet(userId,tweetId)
        # param_2 = obj.getNewsFeed(userId)

```

p.59

Backtracking

NeetCode 150 · Not In 331 · 2 questions

#40 Combination Sum II

MED

Time Complexity: $O(2^n \cdot k)$ in the worst case, where n is the number of candidates and k is the average length of a combination. Space Complexity: $O(k)$ for the recursion stack and current combination (not including the space occupied by the output).

Key Insights

- Sort the candidates first to easily skip duplicates.
- Use backtracking to explore all possible combinations.
- Avoid selecting the same number (at the same recursive level) to avoid duplicate combinations.
- Terminate recursion when the running sum exceeds the target.
- Use indices rather than reusing elements to ensure each number is used at most once.

Brute Force (Python)

```
# Brute - generate all subsets, filter those summing to target
from itertools import combinations
class Solution:
    def combinationSum2(self, candidates: List, target: int) -> List:
        res = set()
        candidates.sort()
        for r in range(1, len(candidates) + 1):
            for combo in combinations(candidates, r):
                if sum(combo) == target: res.add(combo)
        return [list(c) for c in res]
```

Optimal (Python)

```
# Optimal - backtracking with sort + duplicate skip
class Solution:
    def combinationSum2(self, candidates: List, target: int) -> List:
        candidates.sort(); res = []
        def backtrack(start, remain, path):
            if remain == 0: res.append(list(path)); return
            for i in range(start, len(candidates)):
                if candidates[i] > remain: break
                if i > start and candidates[i] == candidates[i-1]:
                    continue
                path.append(candidates[i])
                backtrack(i + 1, remain - candidates[i], path)
                path.pop()
        backtrack(0, target, [])
        return res
```

Best Answers — All Sites (Python)

- [WalkCC](#)

```
class Solution:
    def combinationSum2(self, candidates: List[int],
                        target: int) -> List[List[int]]:

        ans = []

        def dfs(s: int, target: int, path: List[int]) -> None:
            if target < 0:
                return
            if target == 0:
                ans.append(path.copy())
                return

            for i in range(s, len(candidates)):
                if i > s and candidates[i] == candidates[i - 1]:
                    continue
                path.append(candidates[i])
                dfs(i + 1, target - candidates[i], path)
                path.pop()

        candidates.sort()
        dfs(0, target, [])
        return ans
```

- [LeetDoocs](#)

p.61

p.62

p.63

```
class Solution:
    def combinationSum2(self, candidates: List[int], target: int) -> List[List[int]]:
        def dfs(i: int, s: int):
            if s == 0:
                ans.append(t[:])
                return
            if i >= len(candidates) or s < candidates[i]:
                return
            for j in range(i, len(candidates)):
                if j > i and candidates[j] == candidates[j - 1]:
                    continue
                t.append(candidates[j])
                dfs(j + 1, s - candidates[j])
                t.pop()

        candidates.sort()
        ans = []
        t = []
        dfs(0, target)
        return ans
```

- [SimplyLeet](#)

```
# Python Solution for Combination Sum II

def combinationSum2(candidates, target):
    # Sort candidates to handle duplicates easily.
    candidates.sort()
    results = []

    def backtrack(start, current_combination, current_sum):
        # If the current sum equals target, add a copy of the current
        # combination to results.
        if current_sum == target:
            results.append(list(current_combination))
            return
        # Iterate over possible candidates starting from index 'start'.
        for i in range(start, len(candidates)):
            # Skip duplicate elements.
            if i > start and candidates[i] == candidates[i - 1]:
                continue
            new_sum = current_sum + candidates[i]
            # If the new sum exceeds target, break the loop.
            if new_sum > target:
                break
            # Include the candidate and recurse.

            current_combination.append(candidates[i])
            backtrack(i + 1, current_combination, new_sum)
            # Backtrack: remove the last element added.
            current_combination.pop()

    backtrack(0, [], 0)
    return results
```

```
# Example usage:
print(combinationSum2([10,1,2,7,6,1,5], 8))
```

• [LC.ca](#)

```
class Solution:
    def combinationSum2(self, candidates: List[int], target: int) -> List[List[int]]:
        def dfs(i, s):
            if s > target:
                return
            if s == target:
                ans.append(t.copy())
                return
            for j in range(i, len(candidates)):
                # or: if i == j or candidates[j] != candidates[j - 1]
                if j > i and candidates[j] == candidates[j - 1]:
                    continue
                t.append(candidates[j])
                dfs(j + 1, s + candidates[j])
                t.pop()

        ans = []
        candidates.sort()
        t = []
        dfs(0, 0)
        return ans
```

p.64

p.65

p.66

```
#####

class Solution(object):
    def combinationSum2(self, candidates, target):
        """
        :type candidates: List[int]
        :type target: int
        :rtype: List[List[int]]
        """

        def dfs(nums, target, start, visited, path, res):
            if target == 0:
                res.append(path + [])
                return

            for i in range(start, len(nums)):
                if i > start and nums[i] == nums[i - 1]:
                    continue
                if target - nums[i] < 0:
                    return 0
                if i not in visited:
                    visited.add(i)

                    path.append(nums[i])
                    dfs(nums, target - nums[i], i + 1, visited, path, res)
                    path.pop()
                    visited.discard(i)

            candidates.sort()
            res = []
            visited = set([])
            dfs(candidates, target, 0, visited, [], res)
            return res
```

p.67

Backtracking

#131 Palindrome Partitioning

MED

Time Complexity: $O(n * 2^n)$ in the worst case, due to exploring all possible partitions and checking palindromes. Space Complexity: $O(n)$ for recursion stack and temporary storage for current partition.

Key Insights

- Use backtracking to explore all possible ways to partition the string.
- At each step, check whether the current substring is a palindrome.
- If it is, include that substring in the current partition and recursively partition the rest of the string.
- The limit on the string length (at most 16 characters) allows an exponential backtracking approach.
- Optimizations like caching palindrome checks can be used, though the simple approach is typically sufficient.

Brute Force (Python)

```
# Brute Force - backtrack with inline palindrome check
class Solution:
    def partition(self, s: str) -> list:
        res = []
        def backtrack(start, path):
            if start == len(s): res.append(list(path)); return
            for end in range(start + 1, len(s) + 1):
                part = s[start:end]
                if part == part[::-1]:
                    path.append(part); backtrack(end, path); path.pop()
            backtrack(0, [])
        return res
```

Optimal (Python)

p.68

```
# Optimal - precompute palindrome DP table, then backtrack
class Solution:
    def partition(self, s: str) -> list:
        n = len(s)
        dp = [[False]*n for _ in range(n)]
        for i in range(n): dp[i][i] = True
        for length in range(2, n+1):
            for i in range(n - length + 1):
                j = i + length - 1
                dp[i][j] = (s[i] == s[j]) if length == 2 else (s[i] == s[j] and dp[i+1][j-1])
            res = []
            def backtrack(start, path):
                if start == n: res.append(list(path)); return
                for end in range(start, n):
                    if dp[start][end]:
                        path.append(s[start:end+1]); backtrack(end+1, path); path.pop()
            backtrack(0, [])
            return res
```

Best Answers — All Sites (Python)

- [WalkCC](#)

p.69

```
class Solution:
    def partition(self, s: str) -> list[list[str]]:
        ans = []

        def isPalindrome(s: str) -> bool:
            return s == s[::-1]

        def dfs(s: str, j: int, path: list[str], ans: list[list[str]]) -> None:
            if j == len(s):
                ans.append(path)
                return

            for i in range(j, len(s)):
                if isPalindrome(s[j: i + 1]):
                    dfs(s, i + 1, path + [s[j: i + 1]], ans)

        dfs(s, 0, [], ans)
        return ans
```

- [LeetDoocs](#)

p.70

```
class Solution:
    def partition(self, s: str) -> List[List[str]]:
        def dfs(i: int):
            if i == n:
                ans.append(t[:])
                return
            for j in range(i, n):
                if f[i][j]:
                    t.append(s[i : j + 1])
                    dfs(j + 1)
                    t.pop()

            n = len(s)
            f = [[True] * n for _ in range(n)]
            for i in range(n - 1, -1, -1):
                for j in range(i + 1, n):
                    f[i][j] = s[i] == s[j] and f[i + 1][j - 1]

            ans = []
            t = []
            dfs(0)
            return ans
```

- [SimplyLeet](#)

p.71

```
def partition(s):
    # Helper function to check if a given substring is a palindrome
    def is_palindrome(sub):
        return sub == sub[::-1]

    result = [] # final list to store all possible partitions

    # backtracking helper function: current_index points to the
    # starting index of the current search,
    # current_partition stores the partition done so far.
    def backtrack(current_index, current_partition):
        # if we reached the end of the string, add the
        # current_partition to the result list
        if current_index == len(s):
            result.append(list(current_partition))
            return
        # try all possible substrings starting from current_index
        for i in range(current_index, len(s)):
            substring = s[current_index:i+1] # current substring
            # if the substring is a palindrome, add it to
            # current_partition
            if is_palindrome(substring):
                current_partition.append(substring)
                backtrack(i+1, current_partition)
                # backtrack: remove the last added substring
                current_partition.pop()

        backtrack(0, [])
        return result

# Example usage:
print(partition("aab"))
```

- [LC.ca](#)

p.72

```

'''
>>> t = []
>>> t.append(1)
>>> t.append(2)
>>> t.append(3)
>>> t
[1, 2, 3]
>>> t.pop(-1)
3
>>> t
[1, 2]
>>> t.pop()
2
>>> t
[1]

>>> t = [1,2,3]
>>> t.pop(0)
1
>>> t
[2, 3]

```

p.73

```

>>> t = [1,2,3]
>>> t.pop(1)
2
>>> t
[1, 3]
'''

class Solution:
    def partition(self, s: str) -> List[List[str]]:
        ans = []
        n = len(s)
        dp = [[False] * n for _ in range(n)]

        for i in range(n - 1, -1, -1):
            for j in range(i, n):
                # <=3 also working, same as <=1
                # i==j, or i+1==j
                dp[i][j] = s[i] == s[j] and (abs(i - j) <= 1 or dp[i + 1][j - 1])

        def dfs(s, i, t):
            nonlocal n # note: still pass OJ without this nonlocal.
            # default is non-local

```

p.74

```

        if i == n:
            ans.append(t.copy())
            return
        for j in range(i, n): # including single char dp[i][i]
            if dp[i][j]:
                t.append(s[i : j + 1])
                dfs(s, j + 1, t)
                t.pop(-1)

        dfs(s, 0, [])
        return ans

#####

class Solution(object): # iteration, real dp
    def partition(self, s):
        '''
        :type s: str
        :rtype: List[List[str]]
        '''

        pal = [[False for i in range(0, len(s))] for j in range(0, len(s))]
        ans = [[[]]] + [[] for _ in range(len(s))] # length is n+1

        for i in range(0, len(s)):
            for j in range(0, i + 1):
                if (s[j] == s[i]) and ((j + 1 > i - 1) or (pal[j + 1][i - 1])):
                    pal[j][i] = True
                    for res in ans[j]:
                        a = res + [s[j:i + 1]]
                        ans[i + 1].append(a)
        return ans[-1]

```

p.75

Graphs

NeetCode 150 · Not In 331 · 1 question

p.76

Graphs

#684 Redundant Connection

MED

Time Complexity: $O(n * \alpha(n))$ where $\alpha(n)$ is the inverse Ackermann function (almost constant in practice). Space Complexity: $O(n)$, for storing the parent array used in the Union Find structure.

Key Insights

- The problem can be solved using a Union Find (Disjoint Set Union) data structure.
- As we iterate over each edge, we check whether the two vertices are already connected.
- If connecting the two vertices would create a cycle (i.e., they already belong to the same set), then the current edge is redundant.
- Alternatives like DFS or BFS are possible, but Union Find offers a more straightforward and efficient solution.

Brute Force (Python)

```

# Brute Force  $O(n^2)$  - remove each edge, check if graph stays connected
from collections import defaultdict, deque

class Solution:
    def findRedundantConnection(self, edges: list) -> list:
        def connected(edges, n):
            g = defaultdict(list)
            for u, v in edges: g[u].append(v); g[v].append(u)
            visited, q = set(), deque([1])
            while q:
                node = q.popleft()
                if node in visited: continue
                visited.add(node); q.extend(g[node])
            return len(visited) == n
        n = len(edges)
        for i in range(n-1, -1, -1):
            if connected(edges[:i] + edges[i+1:], n): return edges[i]

```

Optimal (Python)

p.77

p.78

```
# Optimal - Union-Find with path compression + rank
class Solution:
    def findRedundantConnection(self, edges: list) -> list:
        n = len(edges)
        parent = list(range(n + 1)); rank = [0] * (n + 1)
        def find(x):
            while parent[x] != x: parent[x] = parent[parent[x]]; x = parent[x]
            return x
        def union(x, y):
            rx, ry = find(x), find(y)
            if rx == ry: return False
            if rank[rx] < rank[ry]: rx, ry = ry, rx
            parent[ry] = rx
            if rank[rx] == rank[ry]: rank[rx] += 1
            return True
        for u, v in edges:
            if not union(u, v): return [u, v]
```

Best Answers — All Sites (Python)

• WalkCC

p.79

```
class UnionFind:
    def __init__(self, n: int):
        self.id = list(range(n))
        self.rank = [0] * n

    def unionByRank(self, u: int, v: int) -> bool:
        i = self._find(u)
        j = self._find(v)
        if i == j:
            return False
        if self.rank[i] < self.rank[j]:
            self.id[i] = j
        elif self.rank[i] > self.rank[j]:
            self.id[j] = i
        else:
            self.id[i] = j
            self.rank[j] += 1
        return True

    def _find(self, u: int) -> int:
        if self.id[u] != u:
            self.id[u] = self._find(self.id[u])

        return self.id[u]

class Solution:
    def findRedundantConnection(self, edges: list[list[int]]) -> list[int]:
        uf = UnionFind(len(edges) + 1)

        for edge in edges:
            u, v = edge
            if not uf.unionByRank(u, v):
                return edge
```

p.80

• LeetCode

```
class Solution:
    def findRedundantConnection(self, edges: List[List[int]]) -> List[int]:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(len(edges)))
        for a, b in edges:
            pa, pb = find(a - 1), find(b - 1)
            if pa == pb:
                return [a, b]
            p[pa] = pb
```

p.81

```
class UnionFind:
    __slots__ = "p", "size"

    def __init__(self, n: int):
        self.p: List[int] = list(range(n))
        self.size: List[int] = [1] * n

    def find(self, x: int) -> int:
        if self.p[x] != x:
            self.p[x] = self.find(self.p[x])
        return self.p[x]

    def union(self, a: int, b: int) -> bool:
        pa, pb = self.find(a), self.find(b)
        if pa == pb:
            return False
        if self.size[pa] > self.size[pb]:
            self.p[pb] = pa
            self.size[pa] += self.size[pb]
        else:
            self.p[pa] = pb
            self.size[pb] += self.size[pa]

        return True

class Solution:
    def findRedundantConnection(self, edges: List[List[int]]) -> List[int]:
        uf = UnionFind(len(edges))
        for a, b in edges:
            if not uf.union(a - 1, b - 1):
                return [a, b]
```

• SimplyLeet

p.82

```
# Python solution using Union Find with path compression.

class UnionFind:
    def __init__(self, size):
        # Initialize parent pointers for each node.
        self.parent = list(range(size))

    def find(self, x):
        # Find the root of x with path compression.
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        # Union the sets of x and y.
        rootX = self.find(x)
        rootY = self.find(y)
        if rootX == rootY:
            return False # Cycle detected.
        self.parent[rootY] = rootX
        return True
```

Sheet 14 / 41 · LeetCode NeetCode 32 · Not In 331

```
def findRedundantConnection(edges):
    # Number of nodes is equal to len(edges) because edges were from a tree.
    n = len(edges)
    uf = UnionFind(n + 1) # Account for node labels starting at 1.
    redundant_edge = None
    for edge in edges:
        a, b = edge
        # If union returns False, a cycle is detected.
        if not uf.union(a, b):
            redundant_edge = edge
    return redundant_edge

# Example usage:
edges = [[1,2],[1,3],[2,3]]
print(findRedundantConnection(edges)) # Output: [2, 3]
```

• LC.ca

```
class Solution:
    def findRedundantConnection(self, edges: List[List[int]]) -> List[int]:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        p = list(range(1010))
        for a, b in edges:
            if find(a) == find(b):
                return [a, b]
            p[find(a)] = find(b)
        return []
```

p.83

p.84

Graphs — Advanced

NeetCode 150 · Not In 331 · 4 questions

Graphs — Advanced

#743 Network Delay Time

MED

Time Complexity: $O(E \log V)$ where E is the number of edges and V is the number of nodes. Space Complexity: $O(V + E)$ due to the adjacency list and distance storage.

Key Insights

- This is a single-source shortest path problem on a directed graph.
- Since all edge weights are non-negative, Dijkstra's algorithm is well-suited for this problem.
- Construct an adjacency list for the graph.
- Utilize a min-heap (priority queue) to efficiently select the next node with the smallest cumulative travel time.
- The answer is the maximum travel time among all reachable nodes; if any node is unreachable, return -1 .

Brute Force (Python)

```
# Brute Force  $O(V \cdot E)$  - Bellman-Ford (relax all edges  $V-1$  times)
class Solution:
    def networkDelayTime(self, times: List[List[int]], n: int, k: int) -> int:
        dist = {i: float('inf') for i in range(1, n+1)}; dist[k] = 0
        for _ in range(n - 1):
            for u, v, w in times:
                if dist[u] + w < dist[v]: dist[v] = dist[u] + w
        res = max(dist.values())
        return res if res < float('inf') else -1
```

Optimal (Python)

```
class Solution:
    def networkDelayTime(self, times: List[List[int]], n: int, k: int) -> int:
        graph = [[] for _ in range(n)]

        for u, v, w in times:
            graph[u - 1].append((v - 1, w))

        return self._dijkstra(graph, k - 1)

    def _dijkstra(self, graph: List[List[tuple[int, int]]], src: int) -> int:
        dist = [math.inf] * len(graph)
        dist[src] = 0
        minHeap = [(dist[src], src)] # (d, u)

        while minHeap:
            d, u = heapq.heappop(minHeap)
            if d > dist[u]:
                continue
            for v, w in graph[u]:
                if d + w < dist[v]:
                    dist[v] = d + w

            heapq.heappush(minHeap, (dist[v], v))

        maxDist = max(dist)
        return maxDist if maxDist != math.inf else -1
```

• LeetCode

```
class Solution:
    def networkDelayTime(self, times: List[List[int]], n: int, k: int) -> int:
        g = [[inf] * n for _ in range(n)]
        for u, v, w in times:
            g[u - 1][v - 1] = w
        dist = [inf] * n
        dist[k - 1] = 0
        vis = [False] * n
        for _ in range(n):
            t = -1
            for j in range(n):
                if not vis[j] and (t == -1 or dist[t] > dist[j]):
                    t = j
            vis[t] = True
            for j in range(n):
                dist[j] = min(dist[j], dist[t] + g[t][j])
        ans = max(dist)
        return -1 if ans == inf else ans
```

```
# Optimal  $O(E \log V)$  - Dijkstra with min-heap
import heapq
from collections import defaultdict
class Solution:
    def networkDelayTime(self, times: List, n: int, k: int) -> int:
        graph = defaultdict(list)
        for u, v, w in times: graph[u].append((v, w))
        dist = {}; heap = [(0, k)]
        while heap:
            d, node = heapq.heappop(heap)
            if node in dist: continue
            dist[node] = d
            for nei, w in graph[node]:
                if nei not in dist: heapq.heappush(heap, (d + w, nei))
        return max(dist.values()) if len(dist) == n else -1
```

Best Answers — All Sites (Python)

• WalkCC

p.85

p.86

p.87

p.88

Sheet 15 / 41 · LeetCode NeetCode 32 · Not In 331

p.89

p.90

```
import heapq

def networkDelayTime(times, n, k):
    # Build the graph as an adjacency list
    graph = {i: [] for i in range(1, n + 1)}
    for u, v, w in times:
        graph[u].append((v, w))

    # Initialize distances: infinity for all nodes except the starting
    # node k with distance 0
    dist = {i: float('inf') for i in range(1, n + 1)}
    dist[k] = 0
    # Use a heap as a priority queue storing (distance, node) pairs
    heap = [(0, k)]

    while heap:
        current_dist, node = heapq.heappop(heap)
        # If we have already found a better path, skip
        if current_dist > dist[node]:
            continue
        # Update the path for neighbors
        for neighbor, weight in graph[node]:
            new_dist = current_dist + weight

            if new_dist < dist[neighbor]:
                dist[neighbor] = new_dist
                heapq.heappush(heap, (new_dist, neighbor))

    # Maximum distance among all nodes represents the time taken for
    # the signal to reach every node
    max_time = max(dist.values())
    return max_time if max_time != float('inf') else -1

# Example usage:
print(networkDelayTime([[2,1,1],[2,3,1],[3,4,1]], 4, 2))
```

p.91

• LC.ca

```
class Solution:
    def networkDelayTime(self, times: List[List[int]], n: int, k: int)
    -> int:
        INF = 0x3F3F
        g = defaultdict(list)
        for u, v, w in times:
            g[u - 1].append((v - 1, w))
        dist = [INF] * n
        dist[k - 1] = 0
        q = [(0, k - 1)]
        while q:
            _, u = heappop(q)
            for v, w in g[u]:
                if dist[v] > dist[u] + w:
                    dist[v] = dist[u] + w
                    heappush(q, (dist[v], v))
        ans = max(dist)
        return -1 if ans == INF else ans
```

p.92

#332 Reconstruct Itinerary

HAR

Time Complexity: $O(E \log E)$, where E is the number of tickets, due to the sorting of edges. Space Complexity: $O(E)$, to store the graph and recursion stack.

Key Insights

- Model the tickets as a directed graph where airports are nodes.
- The problem is akin to finding an Eulerian path through the graph.
- Use Depth-First Search (DFS) with Hierholzer's algorithm to construct the itinerary.
- To ensure lexical order, sort the destination lists in reverse order so that the smallest lexical destination can be popped during DFS.
- Build the itinerary in reverse during backtracking and then reverse it to get the final order.

Brute Force (Python)

```
# DFS backtracking (iterative Hierholzer)
from collections import defaultdict
class Solution:
    def findItinerary(self, tickets: list) -> list:
        graph = defaultdict(list)
        for src, dst in sorted(tickets): graph[src].append(dst)
        result = []; stack = ["JFK"]
        while stack:
            while graph[stack[-1]]:
                stack.append(graph[stack[-1]].pop(0))
            result.append(stack.pop())
        return result[::-1]
```

Optimal (Python)

p.93

```
# Optimal  $O(E \log E)$  - Hierholzer with sorted adjacency lists
from collections import defaultdict
class Solution:
    def findItinerary(self, tickets: list) -> list:
        graph = defaultdict(list)
        for src, dst in tickets: graph[src].append(dst)
        for src in graph: graph[src].sort(reverse=True)
        result = []; stack = ["JFK"]
        while stack:
            while graph[stack[-1]]:
                stack.append(graph[stack[-1]].pop())
            result.append(stack.pop())
        return result[::-1]
```

Best Answers — All Sites (Python)

• WalkCC

```
class Solution:
    def findItinerary(self, tickets: list[list[str]]) -> list[str]:
        ans = []
        graph = collections.defaultdict(list)

        for a, b in reversed(sorted(tickets)):
            graph[a].append(b)

        def dfs(u: str) -> None:
            while u in graph and graph[u]:
                dfs(graph[u].pop())
            ans.append(u)

        dfs('JFK')
        return ans[::-1]
```

• LeetDoocs

p.94

```
class Solution:
    def findItinerary(self, tickets: List[List[str]]) -> List[str]:
        def dfs(f: str):
            while g[f]:
                dfs(g[f].pop())
            ans.append(f)

        g = defaultdict(list)
        for f, t in sorted(tickets, reverse=True):
            g[f].append(t)
        ans = []
        dfs("JFK")
        return ans[::-1]
```

• SimplyLeet

```
def findItinerary(tickets):
    # Build the graph as an adjacency list.
    graph = {}
    for src, dst in tickets:
        if src not in graph:
            graph[src] = []
        graph[src].append(dst)
    # Sort the destination lists in reverse lexicographical order.
    for src in graph:
        graph[src].sort(reverse=True)

    itinerary = []

    def dfs(airport):
        # Explore all flights out of the current airport.
        while airport in graph and graph[airport]:
            next_dest = graph[airport].pop()
            dfs(next_dest)
        itinerary.append(airport)

    # Start DFS from 'JFK'
    dfs("JFK")

    # The itinerary is constructed in reverse.
    return itinerary[::-1]

# Example usage:
tickets = [["JFK","SFO"], ["JFK","ATL"], ["SFO","ATL"], ["ATL","JFK"],
["ATL","SFO"]]
print(findItinerary(tickets))
```

• LC.ca

p.96

p.95

```
class Solution:
    def findItinerary(self, tickets: List[List[str]]) -> List[str]:
        graph = defaultdict(list)

        for src, dst in sorted(tickets, reverse=True):
            graph[src].append(dst)

        itinerary = []

        def dfs(airport):
            while graph[airport]:
                dfs(graph[airport].pop())
            itinerary.append(airport)

        dfs("JFK")

        return itinerary[::-1]
```

p.97

Graphs — Advanced

#1584 Min Cost to Connect All Points MED

Time Complexity: $O(n^2)$ Space Complexity: $O(n)$

Key Insights

- The problem can be reinterpreted as finding a Minimum Spanning Tree (MST) on a complete graph.
- Manhattan distance is used to calculate the weight/cost between every pair of vertices.
- Prim's or Kruskal's algorithm can be applied to solve the MST problem.
- A direct construction of the complete graph is unnecessary; distances can be computed on the fly, which is efficient given the problem constraints.

Brute Force (Python)

```
# Brute Force - Kruskal: generate all edges, sort, union-find
class Solution:
    def minCostConnectPoints(self, points: List) -> int:
        n = len(points); edges = []
        for i in range(n):
            for j in range(i+1, n):
                d = abs(points[i][0]-points[j][0]) +
                abs(points[i][1]-points[j][1])
                edges.append((d, i, j))
        edges.sort(); parent = list(range(n))
        def find(x):
            while parent[x] != x: parent[x] = parent[parent[x]]; x =
            parent[x]
            return x
        cost = used = 0
        for d, u, v in edges:
            ru, rv = find(u), find(v)
            if ru != rv: parent[ru] = rv; cost += d; used += 1
            if used == n-1: break
        return cost
```

Optimal (Python)

p.99

```
# Optimal  $O(n^2 \log n)$  - Prim's with min-heap
import heapq
class Solution:
    def minCostConnectPoints(self, points: List) -> int:
        n = len(points); visited = set(); heap = [(0, 0)]; total = 0
        while len(visited) < n:
            cost, i = heapq.heappop(heap)
            if i in visited: continue
            visited.add(i); total += cost
            for j in range(n):
                if j not in visited:
                    d = abs(points[i][0]-points[j][0]) +
                    abs(points[i][1]-points[j][1])
                    heapq.heappush(heap, (d, j))
        return total
```

Best Answers — All Sites (Python)

• WalkCC

p.100

```
class Solution:
    def minCostConnectPoints(self, points: List[int]) -> int:
        # dist[i] := the minimum distance to connect the points[i]
        dist = [math.inf] * len(points)
        ans = 0

        for i in range(len(points) - 1):
            for j in range(i + 1, len(points)):
                # Try to connect the points[i] with the points[j].
                dist[j] = min(dist[j], abs(points[i][0] - points[j][0]) +
                abs(points[i][1] - points[j][1]))
            # Swap the points[j] (the point with the mnimum distance) with
            the
            # points[i + 1].
            if dist[j] < dist[i + 1]:
                points[j], points[i + 1] = points[i + 1], points[j]
                dist[j], dist[i + 1] = dist[i + 1], dist[j]
            ans += dist[i + 1]

        return ans
```

• LeetCode

Sheet 17 / 41 · LeetCode NeetCode 32 · Not In 331 p.101

```
class Solution:
    def minCostConnectPoints(self, points: List[List[int]]) -> int:
        n = len(points)
        g = [[0] * n for _ in range(n)]
        dist = [inf] * n
        vis = [False] * n
        for i, (x1, y1) in enumerate(points):
            for j in range(i + 1, n):
                x2, y2 = points[j]
                t = abs(x1 - x2) + abs(y1 - y2)
                g[i][j] = g[j][i] = t

        dist[0] = 0
        ans = 0
        for _ in range(n):
            i = -1
            for j in range(n):
                if not vis[j] and (i == -1 or dist[j] < dist[i]):
                    i = j
            vis[i] = True
            ans += dist[i]
            for j in range(n):
                if not vis[j]:
                    dist[j] = min(dist[j], g[i][j])

        return ans
```

p.102

```
class Solution:
    def minCostConnectPoints(self, points: List[List[int]]) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(points)
        g = []
        for i, (x1, y1) in enumerate(points):
            for j in range(i + 1, n):
                x2, y2 = points[j]
                t = abs(x1 - x2) + abs(y1 - y2)
                g.append((t, i, j))
        p = list(range(n))
        ans = 0
        for cost, i, j in sorted(g):
            pa, pb = find(i), find(j)
            if pa == pb:
                continue
            p[pa] = pb
            ans += cost

        n -= 1
        if n == 1:
            break
        return ans
```

• SimplyLeet

p.103

```
# Python solution using Prim's algorithm

class Solution:
    def minCostConnectPoints(self, points):
        # number of points
        n = len(points)
        # initialize cost for each point as infinity, except the first
        point

        minCost = [float('inf')] * n
        minCost[0] = 0 # start from the first point

        # keep track of which points are included in the MST
        inMST = [False] * n

        # total cost of connecting all points
        total_cost = 0

        for _ in range(n):
            # select the next point with the smallest connection cost
            curr = -1
            curr_cost = float('inf')
            for i in range(n):
                if not inMST[i] and minCost[i] < curr_cost:
```

p.104

```
            curr = i
            curr_cost = minCost[i]

        # add the cost of the chosen point
        total_cost += curr_cost
        inMST[curr] = True

        # update the connection costs for the remaining points
        for next_point in range(n):
            if not inMST[next_point]:
                # compute the Manhattan distance between the
                current and next point
                (x1, y1), (x2, y2) = points[curr],
                points[next_point]
                dist = abs(x1 - x2) + abs(y1 - y2)
                if dist < minCost[next_point]:
                    minCost[next_point] = dist

        return total_cost

# Example usage:
# sol = Solution()
# print(sol.minCostConnectPoints([[0,0],[2,2],[3,10],[5,2],[7,0]]))
```

• LC.ca

p.105

```
class Solution:
    def minCostConnectPoints(self, points: List[List[int]]) -> int:
        n = len(points)
        g = [[0] * n for _ in range(n)]
        dist = [inf] * n
        vis = [False] * n
        for i, (x1, y1) in enumerate(points):
            for j in range(i + 1, n):
                x2, y2 = points[j]
                t = abs(x1 - x2) + abs(y1 - y2)
                g[i][j] = g[j][i] = t

        dist[0] = 0
        ans = 0
        for _ in range(n):
            i = -1
            for j in range(n):
                if not vis[j] and (i == -1 or dist[j] < dist[i]):
                    i = j
            vis[i] = True
            ans += dist[i]
            for j in range(n):
                if not vis[j]:
                    dist[j] = min(dist[j], g[i][j])

        return ans
```

p.106

Graphs — Advanced

#778 Swim in Rising Water

HAR

Time Complexity: $O(n^2 \log(n^2))$ which simplifies to $O(n^2 \log n)$ since each cell is processed once and we use a priority queue. Space Complexity: $O(n^2)$ due to the visited matrix and the priority queue.

Key Insights

- The problem can be viewed as finding a path from the top-left to bottom-right where the cost is the maximum elevation encountered on that path.
- A greedy approach using Dijkstra's algorithm (or a min-heap based BFS) ensures that we always consider the next move with the smallest current elevation requirement.
- Once the destination is reached, the maximum elevation along that path is the minimum time required.
- Alternative approaches include binary search on time combined with a DFS/BFS check to see if a valid path exists.

Brute Force (Python)

```
# Brute Force – binary search on answer t; BFS to check reachability
from collections import deque

class Solution:
    def swimInWater(self, grid: list) -> int:
        n = len(grid)
        def can_reach(t):
            if grid[0][0] > t: return False
            visited = {(0,0)}; q = deque([(0,0)])
            while q:
                r, c = q.popleft()
                if r == n-1 and c == n-1: return True
                for dr, dc in [(1,0), (-1,0), (0,1), (0,-1)]:
                    nr, nc = r+dr, c+dc
                    if 0 <= nr < n and 0 <= nc < n and (nr,nc) not in visited
                    and grid[nr][nc] <= t:
                        visited.add((nr,nc)); q.append((nr,nc))
            return False
        lo, hi = grid[0][0], n*n-1
        while lo < hi:
            mid = (lo+hi)//2
            if can_reach(mid): hi = mid
            else: lo = mid+1
        return lo
```

Optimal (Python)

```
# Optimal - Dijkstra with min-max cost
import heapq
class Solution:
    def swimInWater(self, grid: list) -> int:
        n = len(grid)
        dist = [[float('inf')]*n for _ in range(n)]
        dist[0][0] = grid[0][0]; heap = [(grid[0][0], 0, 0)]
        while heap:
            cost, r, c = heapq.heappop(heap)
            if cost > dist[r][c]: continue
            if r == n-1 and c == n-1: return cost
            for dr, dc in [(1,0),(-1,0),(0,1),(0,-1)]:
                nr, nc = r+dr, c+dc
                if 0 <= nr < n and 0 <= nc < n:
                    new_cost = max(cost, grid[nr][nc])
                    if new_cost < dist[nr][nc]:
                        dist[nr][nc] = new_cost
                        heapq.heappush(heap, (new_cost, nr, nc))
        return dist[n-1][n-1]
```

Best Answers — All Sites (Python)

• LeetCode

p.109

```
class Solution:
    def swimInWater(self, grid: List[List[int]]) -> int:
        def find(x: int) -> int:
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(grid)
        m = n * n
        p = list(range(m))
        hi = [0] * m
        for i, row in enumerate(grid):
            for j, h in enumerate(row):
                hi[h] = i * n + j
        dirs = (-1, 0, 1, 0, -1)
        for t in range(m):
            x, y = divmod(hi[t], n)
            for dx, dy in pairwise(dirs):
                nx, ny = x + dx, y + dy
                if 0 <= nx < n and 0 <= ny < n and grid[nx][ny] <= t:
                    p[find(x * n + y)] = find(nx * n + ny)
            if find(0) == find(m - 1):
                return t
        return 0
```

• SimplyLeet

p.110

```
import heapq

def swimInWater(grid):
    n = len(grid)
    # Initialize a visited set to track cells we have processed.
    visited = [[False] * n for _ in range(n)]
    # Min-heap: (current max elevation along the path, row, col)
    min_heap = [(grid[0][0], 0, 0)]
    visited[0][0] = True
    # Directions for moving up, down, left, and right.
    directions = [(1,0), (-1,0), (0,1), (0,-1)]

    while min_heap:
        time, r, c = heapq.heappop(min_heap)
        # If we reach the bottom-right, return the maximum elevation
        # encountered along the path.
        if r == n - 1 and c == n - 1:
            return time
        for dr, dc in directions:
            nr, nc = r + dr, c + dc
            # Check boundaries and if the cell is not already visited.
            if 0 <= nr < n and 0 <= nc < n and not visited[nr][nc]:
                visited[nr][nc] = True

                # The required time is the maximum of the current time
                # and the elevation at the new cell.
                new_time = max(time, grid[nr][nc])
                heapq.heappush(min_heap, (new_time, nr, nc))

    return -1 # Should not reach here if input is as per constraints.

# Example Usage:
grid = [[0,2],[1,3]]
print(swimInWater(grid)) # Output: 3
```

p.111

• LC.ca

```
class Solution:
    def swimInWater(self, grid: List[List[int]]) -> int:
        def find(x):
            if p[x] != x:
                p[x] = find(p[x])
            return p[x]

        n = len(grid)
        p = list(range(n * n))
        hi = [0] * (n * n)
        for i, row in enumerate(grid):
            for j, h in enumerate(row):
                hi[h] = i * n + j
        for t in range(n * n):
            i, j = hi[t] // n, hi[t] % n
            for a, b in [(0, -1), (0, 1), (1, 0), (-1, 0)]:
                x, y = i + a, j + b
                if 0 <= x < n and 0 <= y < n and grid[x][y] <= t:
                    p[find(x * n + y)] = find(hi[t])
            if find(0) == find(n * n - 1):
                return t
        return -1
```

p.112

Dynamic Programming 1D

NeetCode 150 · Not In 331 · 3 questions

p.113

Dynamic Programming 1D

#746 Min Cost Climbing Stairs

EAS

Time Complexity: O(n) Space Complexity: O(n) (can be optimized to O(1) by using only two variables)

Key Insights

- Use Dynamic Programming to solve overlapping subproblems.
- The minimum cost to reach a step is the cost of that step plus the minimum cost of reaching one of the two previous steps.
- The final result is the minimum cost to reach the last or second last step since you can jump from either directly to the top.

Brute Force (Python)

```
# Brute Force O(2^n) - pure recursion without memoisation
class Solution:
    def minCostClimbingStairs(self, cost: list) -> int:
        n = len(cost)
        def dfs(i):
            if i >= n: return 0
            return cost[i] + min(dfs(i+1), dfs(i+2))
        return min(dfs(0), dfs(1))
```

Optimal (Python)

```
# Optimal O(n) time O(1) space - two-variable DP
class Solution:
    def minCostClimbingStairs(self, cost: list) -> int:
        a, b = 0, 0
        for i in range(len(cost) - 1, -1, -1):
            curr = cost[i] + min(a, b); a, b = curr, a
        return min(a, b)
```

p.114

• WalkCC

```
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        cost.append(0)

        for i in range(2, len(cost)):
            cost[i] += min(cost[i - 1], cost[i - 2])

        return cost[-1]
```

• LeetCode

```
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        @cache
        def dfs(i: int) -> int:
            if i >= len(cost):
                return 0
            return cost[i] + min(dfs(i + 1), dfs(i + 2))

        return min(dfs(0), dfs(1))

class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        n = len(cost)
        f = [0] * (n + 1)
        for i in range(2, n + 1):
            f[i] = min(f[i - 2] + cost[i - 2], f[i - 1] + cost[i - 1])
        return f[n]
```

p.115

```
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        f = g = 0
        for i in range(2, len(cost) + 1):
            f, g = g, min(f + cost[i - 2], g + cost[i - 1])
        return g
```

• SimplyLeet

```
# Python implementation with detailed comments

def minCostClimbingStairs(cost):
    n = len(cost) # number of steps
    # dp array to store minimum cost to reach each step
    dp = [0] * n
    dp[0] = cost[0] # cost to start at step 0
    dp[1] = cost[1] # cost to start at step 1

    # Compute minimum cost for each subsequent step
    for i in range(2, n):
        # cost to reach current step is current cost + min cost of the
        # two previous steps
        dp[i] = cost[i] + min(dp[i-1], dp[i-2])

    # The top can be reached from the last or second last step
    return min(dp[n-1], dp[n-2])

# Example usage:
cost = [10, 15, 20]
print(minCostClimbingStairs(cost))
```

• LC.ca

p.116

```
class Solution:
    def minCostClimbingStairs(self, cost: List[int]) -> int:
        f = g = 0
        for i in range(2, len(cost) + 1):
            f, g = g, min(f + cost[i - 2], g + cost[i - 1])
        return g
```

p.117

Dynamic Programming 1D

MED

#213 House Robber II

Time Complexity: O(n), where n is the number of houses. Space Complexity: O(1), as only a constant amount of extra space is used.

Key Insights

- The houses are arranged in a circle, meaning the first and last houses are neighbors.
- Because of the circular arrangement, you cannot rob both the first and last houses.
- The problem can be reduced into two separate linear House Robber problems:
- One considering houses from index 1 to n-1 (exclude the first house).
- The other considering houses from index 0 to n-2 (exclude the last house).
- Use dynamic programming with constant space (using two variables) to compute the solution for a linear array of houses.

Brute Force (Python)

```
# Brute Force O(2^n) - recursive without memoisation
class Solution:
    def rob(self, nums: List) -> int:
        if len(nums) == 1: return nums[0]
        def rob_range(arr):
            def recurse(i):
                if i >= len(arr): return 0
                return max(arr[i] + recurse(i+2), recurse(i+1))
            return recurse(0)
        return max(rob_range(nums[:-1]), rob_range(nums[1:]))
```

Optimal (Python)

p.118

```
# Optimal O(n) time O(1) space - two linear passes
class Solution:
    def rob(self, nums: List) -> int:
        if len(nums) == 1: return nums[0]
        def rob_range(arr):
            a, b = 0, 0
            for n in arr: a, b = b, max(b, a + n)
            return b
        return max(rob_range(nums[:-1]), rob_range(nums[1:]))
```

Best Answers — All Sites (Python)

• WalkCC

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        if not nums:
            return 0
        if len(nums) < 2:
            return nums[0]

        def rob(l: int, r: int) -> int:
            dp1 = 0
            dp2 = 0

            for i in range(l, r + 1):
                temp = dp1
                dp1 = max(dp1, dp2 + nums[i])
                dp2 = temp

            return dp1

        return max(rob(0, len(nums) - 2),
                   rob(1, len(nums) - 1))
```

• LeetCode

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        def _rob(nums):
            f = g = 0
            for x in nums:
                f, g = max(f, g), f + x
            return max(f, g)

        if len(nums) == 1:
            return nums[0]
        return max(_rob(nums[1:]), _rob(nums[:-1]))
```

• SimplyLeet

p.119

p.120

```
# Python solution for House Robber II
```

```
def rob(nums):
    # Helper function to handle the linear House Robber problem.
    def rob_linear(houses):
        prev_max, curr_max = 0, 0
        for money in houses:
            temp = curr_max
            curr_max = max(curr_max, prev_max + money)
            prev_max = temp
        return curr_max

    n = len(nums)
    # When there's only one house, just return its value.
    if n == 1:
        return nums[0]

    # Rob excluding the first house and excluding the last house.
    return max(rob_linear(nums[1:]), rob_linear(nums[:-1]))
```

```
# Test example
print(rob([2, 3, 2])) # Expected output: 3
```

• [LC.ca](#)

p.121

```
class Solution:
    def rob(self, nums: List[int]) -> int:
        def robRange(nums, l, r): # re-use LC-198 solution
            not_rob, rob = 0, nums[l]
            for num in nums[l + 1 : r + 1]: # to include 'r'
                not_rob, rob = rob, max(num + not_rob, rob)
            return rob

        n = len(nums)
        if n == 1:
            return nums[0]
        s1, s2 = robRange(nums, 0, n - 2), robRange(nums, 1, n - 1) # inclusive
        return max(s1, s2)

#####

class Solution:
    def rob(self, nums: List[int]) -> int:
        def _rob(nums):
            f = g = 0
            for x in nums:
                f, g = max(f, g), f + x

            return max(f, g)

        if len(nums) == 1:
            return nums[0]
        return max(_rob(nums[1:]), _rob(nums[:-1]))
```

p.122

Dynamic Programming 1D

#647 Palindromic Substrings

MED

Time Complexity: $O(n^2)$ where n is the length of the string. Space Complexity: $O(1)$ if using the expand-around-center method (ignoring recursion/stack overhead).

Key Insights

- Every individual character is a palindrome.
- Palindromes can be expanded from a center point; centers can be single characters (odd length) or between two characters (even length).
- The "expand around center" technique efficiently counts palindromes in $O(n^2)$ time.
- Dynamic programming could also be used but requires extra space and is more complex to implement.

Brute Force (Python)

```
# Brute Force  $O(n^3)$  - check every substring
class Solution:
    def countSubstrings(self, s: str) -> int:
        count = 0
        for i in range(len(s)):
            for j in range(i+1, len(s)+1):
                sub = s[i:j]
                if sub == sub[::-1]: count += 1
        return count
```

Optimal (Python)

p.123

```
# Optimal  $O(n^2)$  - expand around every center
class Solution:
    def countSubstrings(self, s: str) -> int:
        def expand(l, r):
            count = 0
            while l >= 0 and r < len(s) and s[l] == s[r]:
                count += 1; l -= 1; r += 1
            return count

        total = 0
        for i in range(len(s)):
            total += expand(i, i) # odd
            total += expand(i, i+1) # even
        return total
```

Best Answers — All Sites (Python)

• [WalkCC](#)

p.124

```
class Solution:
    def countSubstrings(self, s: str) -> int:
        def extendPalindromes(l: int, r: int) -> int:
            count = 0

            while l >= 0 and r < len(s) and s[l] == s[r]:
                count += 1
                l -= 1
                r += 1

            return count

        ans = 0

        for i in range(len(s)):
            ans += extendPalindromes(i, i)
            ans += extendPalindromes(i, i + 1)

        return ans
```

• [LeetDoocs](#)

```
class Solution:
    def countSubstrings(self, s: str) -> int:
        ans, n = 0, len(s)
        for k in range(n * 2 - 1):
            i, j = k // 2, (k + 1) // 2
            while ~i and j < n and s[i] == s[j]:
                ans += 1
                i, j = i - 1, j + 1
        return ans
```

```
class Solution:
    def countSubstrings(self, s: str) -> int:
        t = '^#' + '#' * len(s) + '$'
        n = len(t)
        p = [0 for _ in range(n)]
        pos, maxRight = 0, 0
        ans = 0
        for i in range(1, n - 1):
            p[i] = min(maxRight - i, p[2 * pos - i]) if maxRight > i
            while t[i - p[i]] == t[i + p[i]]:
                p[i] += 1
            if i + p[i] > maxRight:
                maxRight = i + p[i]
                pos = i
            ans += p[i] // 2
        return ans
```

• [SimplyLeet](#)

p.125

p.126

```
# Define a function to count palindromic substrings in a string
def countSubstrings(s: str) -> int:
    # Initialize the result variable to accumulate count of palindromic substrings
    result = 0

    # Helper function that expands around a center (left, right) and counts palindromes
    def expandAroundCenter(left: int, right: int) -> int:
        count = 0
        # Expand as long as indices are within bounds and characters at left and right are equal
        while left >= 0 and right < len(s) and s[left] == s[right]:
            # Found a palindrome, so increment count
            count += 1
            left -= 1 # Move left pointer outward
            right += 1 # Move right pointer outward
        return count

    # Iterate over each character in the string
    for i in range(len(s)):
        # Count odd-length palindromes (center is i)
        result += expandAroundCenter(i, i)
        # Count even-length palindromes (center is between i and i+1)
        result += expandAroundCenter(i, i + 1)

    return result

# Example usage:
# result = countSubstrings("aaa")
# print(result)
```

• [LC.ca](#)

p.127

```
class Solution:
    def countSubstrings(self, s: str) -> int:
        t = '^#' + '#' + s + '#'
        n = len(t)
        p = [0 for _ in range(n)]
        pos, maxRight = 0, 0
        ans = 0
        for i in range(1, n - 1):
            p[i] = min(maxRight - i, p[2 * pos - i]) if maxRight > i
        else 1
            while t[i - p[i]] == t[i + p[i]]:
                p[i] += 1
            if i + p[i] > maxRight:
                maxRight = i + p[i]
                pos = i
            ans += p[i] // 2
        return ans
```

p.128

Dynamic Programming 2D

NeetCode 150 · Not In 331 · 3 questions

p.129

Dynamic Programming 2D

#518 Coin Change II

MED

Time Complexity: $O(n * \text{amount})$, where n is the number of coins. Space Complexity: $O(\text{amount})$, due to the dp array used for computations.

Key Insights

- Use dynamic programming to build up the number of ways to form every amount up to the target.
- A one-dimensional dp array suffices, where $dp[j]$ represents the number of ways to form sum j .
- When processing each coin, iterate over amounts from the coin value to the target, updating dp.
- Order of iteration ensures that combinations, not permutations, are counted.

Brute Force (Python)

```
# Brute Force  $O(2^{\text{amount}})$  - pure recursion
class Solution:
    def change(self, amount: int, coins: List[int]) -> int:
        def dfs(remaining, idx):
            if remaining == 0: return 1
            if remaining < 0 or idx == len(coins): return 0
            return dfs(remaining - coins[idx], idx) + dfs(remaining, idx + 1)
        return dfs(amount, 0)
```

Optimal (Python)

p.130

```
# Optimal  $O(\text{amount} * n)$  - 1D DP, outer = coins, inner = amounts
class Solution:
    def change(self, amount: int, coins: List[int]) -> int:
        dp = [0] * (amount + 1); dp[0] = 1
        for coin in coins:
            for a in range(coin, amount + 1): dp[a] += dp[a - coin]
        return dp[amount]
```

Best Answers — All Sites (Python)

• WalkCC

```
class Solution:
    def change(self, amount: int, coins: List[int]) -> int:
        dp = [1] + [0] * amount
        for coin in coins:
            for i in range(coin, amount + 1):
                dp[i] += dp[i - coin]
        return dp[amount]
```

• LeetCode

```
class Solution:
    def change(self, amount: int, coins: List[int]) -> int:
        m, n = len(coins), amount
        f = [[0] * (n + 1) for _ in range(m + 1)]
        f[0][0] = 1
        for i, x in enumerate(coins, 1):
            for j in range(n + 1):
                f[i][j] = f[i - 1][j]
                if j >= x:
                    f[i][j] += f[i][j - x]
        return f[m][n]
```

```
class Solution:
    def change(self, amount: int, coins: List[int]) -> int:
        n = amount
        f = [1] + [0] * n
        for x in coins:
            for j in range(x, n + 1):
                f[j] += f[j - x]
        return f[n]
```

• SimplyLeet

```
# Python solution for Coin Change II
def change(amount, coins):
    # Initialize dp array where dp[j] represents the number of ways to form amount j
    dp = [0] * (amount + 1)
    dp[0] = 1 # One way to achieve amount 0 (use no coins)

    # Process each coin
    for coin in coins:
        # Update dp for all amounts from coin to target amount
        for j in range(coin, amount + 1):
            dp[j] += dp[j - coin] # Accumulate ways by using coin
    return dp[amount]
```

```
# Example usage:
print(change(5, [1, 2, 5])) # Output: 4
```

• [LC.ca](#)

p.132

p.131

```
class Solution:
    def change(self, amount: int, coins: List[int]) -> int:
        dp = [0] * (amount + 1)
        dp[0] = 1
        for coin in coins:
            for j in range(coin, amount + 1):
                dp[j] += dp[j - coin]
        return dp[-1]
```

p.133

Dynamic Programming 2D

#494 Target Sum

MED

Time Complexity: $O(n * \text{subsetSum})$, where n is the length of the `nums` array and `subsetSum = (target + totalSum)/2`. Space Complexity: $O(\text{subsetSum})$, as we use a one-dimensional DP array.

Key Insights

- The problem can be converted into a subset sum problem by considering two groups: numbers with a '+' sign and numbers with a '-' sign.
- Let the sum of numbers with a '+' sign be `sum(P)` and with a '-' sign be `sum(N)`. Then the problem reduces to solving:
 - `sum(P) - sum(N) = target`
 - and `sum(P) + sum(N) = totalSum`, where `totalSum` is the sum of all numbers in `nums`.
- From these equations, we derive `sum(P) = (target + totalSum) / 2`.
- We only proceed if `(target + totalSum)` is even and not negative; otherwise, there are no valid expressions.
- Dynamic programming is used to count the number of subsets that sum to `sum(P)`.

Brute Force (Python)

```
# Brute Force  $O(2^n)$  - try all +/- assignments
class Solution:
    def findTargetSumWays(self, nums: List[int], target: int) -> int:
        def dfs(i, curr):
            if i == len(nums): return 1 if curr == target else 0
            return dfs(i+1, curr+nums[i]) + dfs(i+1, curr-nums[i])
        return dfs(0, 0)
```

Optimal (Python)

p.134

```
# Optimal  $O(n * \text{total})$  - DP dict (sum -> ways)
class Solution:
    def findTargetSumWays(self, nums: List[int], target: int) -> int:
        dp = {0: 1}
        for n in nums:
            new_dp = {}
            for curr_sum, ways in dp.items():
                new_dp[curr_sum + n] = new_dp.get(curr_sum + n, 0) + ways
                new_dp[curr_sum - n] = new_dp.get(curr_sum - n, 0) + ways
            dp = new_dp
        return dp.get(target, 0)
```

Best Answers — All Sites (Python)

• WalkCC

```
class Solution:
    def findTargetSumWays(self, nums: List[int], target: int) -> int:
        summ = sum(nums)
        if summ < abs(target) or (summ + target) % 2 == 1:
            return 0

        def knapsack(target: int) -> int:
            # dp[i] := the number of ways to sum to i by nums so far
            dp = [1] + [0] * summ

            for num in nums:
                for j in range(summ, num - 1, -1):
                    dp[j] += dp[j - num]

            return dp[target]

        return knapsack((summ + target) // 2)
```

p.135

• LeetCode

```
class Solution:
    def findTargetSumWays(self, nums: List[int], target: int) -> int:
        s = sum(nums)
        if s < target or (s - target) % 2:
            return 0
        m, n = len(nums), (s - target) // 2
        f = [[0] * (n + 1) for _ in range(m + 1)]
        f[0][0] = 1
        for i, x in enumerate(nums, 1):
            for j in range(n + 1):
                f[i][j] = f[i - 1][j]
                if j >= x:
                    f[i][j] += f[i - 1][j - x]
        return f[m][n]
```

• SimplyLeet

p.136

```
# Define a function to compute the number of ways to reach the target sum.
def findTargetSumWays(nums, target):
    # Calculate the total sum of all numbers in the list.
    total_sum = sum(nums)
    # If the target is greater than total_sum, or (target + total_sum) is odd, no solution exists.
    if target > total_sum or (target + total_sum) % 2 == 1:
        return 0

    # Compute the required subset sum.
    subset_sum = (target + total_sum) // 2
    # Initialize the dp array with zeros, with dp[0] set to 1.
    dp = [0] * (subset_sum + 1)
    dp[0] = 1

    # Update the dp array for each number in nums.
    for num in nums:
        # Iterate backwards to ensure each number is only counted once.
        for s in range(subset_sum, num - 1, -1):
            dp[s] += dp[s - num]
    return dp[subset_sum]

# Example usage:
print(findTargetSumWays([1,1,1,1,1], 3)) # Expected output: 5
```

• LC.ca

```
class Solution:
    def findTargetSumWays(self, nums: List[int], target: int) -> int:
        s = sum(nums)
        if s < target or (s - target) % 2 != 0:
            return 0
        n = (s - target) // 2
        dp = [0] * (n + 1)
        dp[0] = 1
        for v in nums:
            for j in range(n, v - 1, -1):
                dp[j] += dp[j - v]
        return dp[-1]
```

p.137

p.138

#97 Interleaving String

MED

Time Complexity: $O(n * m)$ where n = length of $s1$ and m = length of $s2$.
 Space Complexity: $O(n * m)$ with the basic DP solution. (Can be optimized to $O(m)$ with further space reduction.)

Key Insights

- The problem asks if $s3$ can be constructed by preserving the order of characters in $s1$ and $s2$.
- A dynamic programming approach is effective by defining $dp[i][j]$ to represent whether $s3[0 \dots i+j-1]$ is an interleaving of $s1[0 \dots i-1]$ and $s2[0 \dots j-1]$.
- The solution must check that the total lengths match; otherwise, it is immediately false.
- Transitions are based on whether the current character in $s3$ matches the next char in $s1$ or $s2$.
- Space optimization to $O(s2.length)$ is possible by reusing a 1D DP array.

Brute Force (Python)

```
# Brute Force  $O(2^{(m+n)})$  - recursion without memoisation
class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        if len(s1) + len(s2) != len(s3): return False
        def dfs(i, j, k):
            if k == len(s3): return True
            if i < len(s1) and s1[i] == s3[k] and dfs(i+1, j, k+1):
                return True
            if j < len(s2) and s2[j] == s3[k] and dfs(i, j+1, k+1):
                return True
            return False
        return dfs(0, 0, 0)
```

p.139

Optimal (Python)

```
# Optimal  $O(m*n)$  - 2D DP (rolling 1D array)
class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        m, n = len(s1), len(s2)
        if m + n != len(s3): return False
        dp = [False] * (n + 1); dp[0] = True
        for j in range(1, n+1): dp[j] = dp[j-1] and s2[j-1] == s3[j-1]
        for i in range(1, m+1):
            dp[0] = dp[0] and s1[i-1] == s3[i-1]
            for j in range(1, n+1):
                dp[j] = ((dp[j] and s1[i-1] == s3[i+j-1]) or
                        (dp[j-1] and s2[j-1] == s3[i+j-1]))
        return dp[n]
```

Best Answers — All Sites (Python)

- WalkCC

p.140

```
class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        m = len(s1)
        n = len(s2)
        if m + n != len(s3):
            return False

        # dp[i][j] := true if s3[0..i+j] is formed by the interleaving of
        # s1[0..i] and s2[0..j]
        dp = [[False] * (n + 1) for _ in range(m + 1)]
        dp[0][0] = True

        for i in range(1, m + 1):
            dp[i][0] = dp[i - 1][0] and s1[i - 1] == s3[i - 1]

        for j in range(1, n + 1):
            dp[0][j] = dp[0][j - 1] and s2[j - 1] == s3[j - 1]

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                dp[i][j] = (dp[i - 1][j] and s1[i - 1] == s3[i + j - 1] or
                           dp[i][j - 1] and s2[j - 1] == s3[i + j - 1])

        return dp[m][n]
```

- LeetCode

p.141

```
class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        @cache
        def dfs(i: int, j: int) -> bool:
            if i >= m and j >= n:
                return True
            k = i + j
            if i < m and s1[i] == s3[k] and dfs(i + 1, j):
                return True
            if j < n and s2[j] == s3[k] and dfs(i, j + 1):
                return True
            return False

        m, n = len(s1), len(s2)
        if m + n != len(s3):
            return False
        return dfs(0, 0)
```

```
class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        m, n = len(s1), len(s2)
        if m + n != len(s3):
            return False
        f = [[False] * (n + 1) for _ in range(m + 1)]
        f[0][0] = True
        for i in range(m + 1):
            for j in range(n + 1):
                k = i + j - 1
                if i and s1[i - 1] == s3[k]:
                    f[i][j] = f[i - 1][j]
                if j and s2[j - 1] == s3[k]:
                    f[i][j] |= f[i][j - 1]
        return f[m][n]
```

- SimplyLeet

p.142

```
def isInterleave(s1, s2, s3):
    # Check if the combined lengths of s1 and s2 match s3
    if len(s1) + len(s2) != len(s3):
        return False
    # Initialize dp table with dimensions (len(s1)+1) x (len(s2)+1)
    dp = [[False] * (len(s2) + 1) for _ in range(len(s1) + 1)]
    dp[0][0] = True # Base case: empty strings form an empty string
    # Fill first row using characters from s2 only
    for j in range(1, len(s2) + 1):
        dp[0][j] = dp[0][j - 1] and s2[j - 1] == s3[j - 1]
    # Fill first column using characters from s1 only
    for i in range(1, len(s1) + 1):
        dp[i][0] = dp[i - 1][0] and s1[i - 1] == s3[i - 1]
    # Fill the rest of the dp table
    for i in range(1, len(s1) + 1):
        for j in range(1, len(s2) + 1):
            dp[i][j] = (dp[i - 1][j] and s1[i - 1] == s3[i + j - 1]) or
                       (dp[i][j - 1] and s2[j - 1] == s3[i + j - 1])
    return dp[len(s1)][len(s2)]
```

```
# Example usage:
print(isInterleave("aabcc", "dbbca", "aadbcbccac")) # Expected output:
True
```

```
print(isInterleave("aabcc", "dbbca", "aadbcbacc")) # Expected output:
False
```

- LC.ca

```
# 2-d
class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        if len(s1) + len(s2) != len(s3):
            return False

        dp = [[False] * (len(s2) + 1) for _ in range(len(s1) + 1)]
        dp[0][0] = True

        for i in range(1, len(s1) + 1):
            dp[i][0] = s1[i - 1] == s3[i - 1] and dp[i - 1][0]
        for i in range(1, len(s2) + 1):
            dp[0][i] = s2[i - 1] == s3[i - 1] and dp[0][i - 1]

        # fill in dp array
        for i in range(1, len(s1) + 1):
            for j in range(1, len(s2) + 1):
                with_s1 = s1[i - 1] == s3[i + j - 1] and dp[i - 1][j]
                with_s2 = s2[j - 1] == s3[i + j - 1] and dp[i][j - 1]
                dp[i][j] = with_s1 or with_s2

        return dp[-1][-1]
```

p.144

p.143

```
#####

# 1-d
class Solution:
    def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
        m, n = len(s1), len(s2)
        if m + n != len(s3):
            return False
        f = [True] + [False] * n
        for i in range(m + 1):
            for j in range(n + 1):
                k = i + j - 1
                if i:
                    f[j] &= s1[i - 1] == s3[k]
                if j:
                    f[j] |= f[j - 1] and s2[j - 1] == s3[k]
        return f[n]
```

```
#####

class Solution: # dfs
```

p.145

```
def isInterleave(self, s1: str, s2: str, s3: str) -> bool:
    m, n = len(s1), len(s2)
    if m + n != len(s3):
        return False

    @cache
    def dfs(i, j):
        if i == m and j == n:
            return True

        return (
            (i < m and s1[i] == s3[i + j] and dfs(i + 1, j)) or
            (j < n and s2[j] == s3[i + j] and dfs(i, j + 1))
        )

    return dfs(0, 0)
```

p.146

Greedy

NeetCode 150 · Not In 331 · 5 questions

p.147

Greedy

#45 Jump Game II

MED

Time Complexity: $O(n)$ – Single pass through the array. Space Complexity: $O(1)$ – Only a few variables are used for tracking jumps and ranges.

Key Insights

- Utilize a greedy algorithm to make optimal jumps.
- Track the furthest index that can be reached within the current jump range.
- When the current index reaches the end of the current range, a jump is made and the range updates to the furthest reachable index.
- Edge case: if the array has only one element, no jumps are needed.

Brute Force (Python)

```
# Brute Force  $O(n^2)$  – BFS layer by layer
from collections import deque
class Solution:
    def jump(self, nums: list) -> int:
        n = len(nums)
        if n == 1: return 0
        dist = [-1] * n; dist[0] = 0; q = deque([0])
        while q:
            i = q.popleft()
            for j in range(i+1, min(i+nums[i]+1, n)):
                if dist[j] == -1:
                    dist[j] = dist[i] + 1
                    if j == n-1: return dist[j]
            q.append(j)
        return dist[n-1]
```

Optimal (Python)

p.148

```
# Optimal  $O(n)$  – greedy range expansion
class Solution:
    def jump(self, nums: list) -> int:
        jumps = farthest = end = 0
        for i in range(len(nums) - 1):
            farthest = max(farthest, i + nums[i])
            if i == end: jumps += 1; end = farthest
        return jumps
```

Best Answers — All Sites (Python)

• WalkCC

```
class Solution:
    def jump(self, nums: list[int]) -> int:
        ans = 0
        end = 0
        farthest = 0

        # Start an implicit BFS.
        for i in range(len(nums) - 1):
            farthest = max(farthest, i + nums[i])
            if farthest >= len(nums) - 1:
                ans += 1
                break
            if i == end: # Visited all the items on the current level.
                ans += 1 # Increment the level.
                end = farthest # Make the queue size for the next level.

        return ans
```

• LeetCode

```
class Solution:
    def jump(self, nums: List[int]) -> int:
        ans = mx = last = 0
        for i, x in enumerate(nums[:-1]):
            mx = max(mx, i + x)
            if last == i:
                ans += 1
                last = mx
        return ans
```

• SimplyLeet

```
# Function to calculate minimum number of jumps to reach the last index
def jump(nums):
    # If the list has only one element, we're already at the
    destination.
    if len(nums) == 1:
        return 0

    n = len(nums) # Length of the input array
    jumps = 0 # To count the number of jumps required
    currentEnd = 0 # The end of the current range that can be reached
    furthest = 0 # The furthest index that can be reached from the
    current range

    # Iterate till the second last element because we don't need to
    jump from the last element
    for i in range(n - 1):
        # Update the furthest index reachable from index i
        furthest = max(furthest, i + nums[i])
        # If we have reached the end of the current jump range
        if i == currentEnd:
            jumps += 1 # Increase the jump count as we make a jump
            currentEnd = furthest # Update the currentEnd to the
            furthest reachable index
        return jumps

# Example usage:

print(jump([2,3,1,1,4])) # Output: 2
```

• LC.ca

p.151

```
class Solution:
    def jump(self, nums: List[int]) -> int:
        current_reach = next_reach = steps = 0
        # stop at 2nd-to-last, not the last index.
        # because, eg. nums=[1,0] or nums=[0,0],
        # just check the 2nd-to-last then we can decide if able to
        reach end

        # eg. - if input is [0], then 0 step needed
        for i, num in enumerate(nums[:-1]):
            next_reach = max(next_reach, i + num)
            if i == current_reach:
                current_reach = next_reach # update next-reach before
                steps += 1 # in question, guarenteed can reach end. or
                else need more check
                return steps

        #####

class Solution(object):
    def jump(self, nums):
        """
        :type nums: List[int]
        :rtype: int
```

p.152

```
pos = 0
ans = 0
bound = len(nums)
while pos < len(nums) - 1:
    dis = nums[pos]
    farthest = posToFarthest = 0
    for i in range(pos + 1, min(pos + dis + 1, bound)):
        canReach = i + nums[i]
        if i == len(nums) - 1:
            return ans + 1
        if canReach > farthest:
            farthest = canReach
            posToFarthest = i
    ans += 1
    pos = posToFarthest
return ans
```

p.153

Greedy

#846 Hand of Straights

MED

Time Complexity: $O(n \log n)$ due to sorting the unique keys from the hand. Space Complexity: $O(n)$ for storing the frequency count of the cards.

Key Insights

- The total number of cards must be divisible by groupSize.
- Use a frequency count (hash table or map) to record the occurrence of each card.
- Always start forming groups from the smallest available card to maintain the consecutive order.
- For each card, if its count is non-zero, check that the subsequent groupSize-1 cards exist with at least as many counts.
- Decrease the counts accordingly for each valid consecutive group.

Brute Force (Python)

```
# Brute Force  $O(n^2)$  - repeatedly find and remove groups
class Solution:
    def isNStraightHand(self, hand: List, groupSize: int) -> bool:
        if len(hand) % groupSize: return False
        hand = sorted(hand)
        while hand:
            start = hand[0]
            for i in range(groupSize):
                try: hand.remove(start + i)
                except ValueError: return False
            return True
```

Optimal (Python)

p.154

```
# Optimal  $O(n \log n)$  - Counter + sorted keys
from collections import Counter
class Solution:
    def isNStraightHand(self, hand: List, groupSize: int) -> bool:
        if len(hand) % groupSize: return False
        count = Counter(hand)
        for card in sorted(count):
            n = count[card]
            if n > 0:
                for i in range(groupSize):
                    count[card + i] -= n
                    if count[card + i] < 0: return False
        return True
```

Best Answers — All Sites (Python)

• WalkCC

```
class Solution:
    def isNStraightHand(self, hand: List[int], groupSize: int) -> bool:
        count = collections.Counter(hand)

        for start in sorted(count):
            value = count[start]
            if value > 0:
                for i in range(start, start + groupSize):
                    count[i] -= value
                    if count[i] < 0:
                        return False
        return True
```

• LeetCode

```
class Solution:
    def isNStraightHand(self, hand: List[int], groupSize: int) -> bool:
        if len(hand) % groupSize:
            return False
        cnt = Counter(hand)
        for x in sorted(hand):
            if cnt[x]:
                for y in range(x, x + groupSize):
                    if cnt[y] == 0:
                        return False
                    cnt[y] -= 1
        return True
```

• SimplyLeet

p.155

p.156

```
# Python solution using collections.Counter for frequency mapping
from collections import Counter

def isNStraightHand(hand, groupSize):
    # Check whether total cards can be divided into groups of groupSize
    if len(hand) % groupSize != 0:
        return False

    # Create frequency map of cards
    card_count = Counter(hand)

    # Process sorted keys to ensure we always start with the smallest
    # available card
    for card in sorted(card_count.keys()):
        # If current count is greater than 0, try to form a consecutive
        # group starting from card
        count = card_count[card]
        if count > 0:
            for i in range(groupSize):
                # Check if consecutive card exists in sufficient
                # quantity
                if card_count[card + i] < count:
                    return False
                # Deduct the frequency for the current group
                card_count[card + i] -= count

    return True
```

```
# Example usage:
# print(isNStraightHand([1,2,3,6,2,3,4,7,8], 3)) # Expected output: True
```

• [LC.ca](#)

p.157

```
class Solution:
    def isNStraightHand(self, hand: List[int], groupSize: int) -> bool:
        cnt = Counter(hand)
        for v in sorted(hand):
            if cnt[v]:
                for x in range(v, v + groupSize):
                    if cnt[x] == 0:
                        return False
                    cnt[x] -= 1
                    if cnt[x] == 0:
                        cnt.pop(x)

        return True
```

p.158

Greedy

#1899 Merge Triplets to Form Target Triplet

MED

Time Complexity: $O(n)$, where n is the number of triplets. Space Complexity: $O(1)$ extra space.

Key Insights

- Only consider triplets that are "valid" in the sense that none of its values exceed the corresponding target value.
- For each component of the target, there must be at least one valid triplet that provides that maximum value.
- By taking the element-wise maximum across all valid triplets, if we can recreate the target, then the target can be formed using a series of allowed operations.

Brute Force (Python)

```
# Brute Force  $O(2^n)$  - try all subsets of triplets
class Solution:
    def mergeTriplets(self, triplets: List[List[int]], target: List[int]) -> bool:
        for mask in range(1, 1 << len(triplets)):
            merged = [0, 0, 0]
            for i in range(len(triplets)):
                if mask & (1 << i): merged = [max(merged[j],
                    triplets[i][j]) for j in range(3)]
                if merged == target: return True
            return False
```

Optimal (Python)

p.159

```
# Optimal  $O(n)$  - filter invalid triplets, take element-wise max
class Solution:
    def mergeTriplets(self, triplets: List[List[int]], target: List[int]) -> bool:
        res = [0, 0, 0]
        for t in triplets:
            if t[0] <= target[0] and t[1] <= target[1] and t[2] <=
target[2]:
                res = [max(res[i], t[i]) for i in range(3)]
        return res == target
```

Best Answers — All Sites (Python)

• [WalkCC](#)

```
class Solution:
    def mergeTriplets(self, triplets: List[List[int]], target: List[int])
-> bool:
        merged = [0] * len(target)

        for triplet in triplets:
            if all(a <= b for a, b in zip(triplet, target)):
                for i in range(3):
                    merged[i] = max(merged[i], triplet[i])

        return merged == target
```

• [LeetDoocs](#)

p.160

```
class Solution:
    def mergeTriplets(self, triplets: List[List[int]], target:
List[int]) -> bool:
        x, y, z = target
        d = e = f = 0
        for a, b, c in triplets:
            if a <= x and b <= y and c <= z:
                d = max(d, a)
                e = max(e, b)
                f = max(f, c)
        return [d, e, f] == target

    object Solution {
        def mergeTriplets(triplets: Array[Array[Int]], target: Array[Int]):
Boolean = {
            val Array(x, y, z) = target
            var (d, e, f) = (0, 0, 0)

            for (Array(a, b, c) <- triplets) {
                if (a <= x && b <= y && c <= z) {
                    d = d.max(a)
                    e = e.max(b)
                    f = f.max(c)
                }
            }

            d == x && e == y && f == z
        }
    }
```

• [SimplyLeet](#)

```
def mergeTriplets(triplets, target):
    # Initialize candidate triplet with zeros
    candidate = [0, 0, 0]
    # Iterate over every triplet in the input list
    for trip in triplets:
        # Only consider triplets where each element is <= corresponding
        # target element
        if trip[0] <= target[0] and trip[1] <= target[1] and trip[2] <=
target[2]:
            # Update candidate by taking the element-wise maximum
            candidate[0] = max(candidate[0], trip[0])
            candidate[1] = max(candidate[1], trip[1])
            candidate[2] = max(candidate[2], trip[2])
        # If the candidate matches target, return True; otherwise, return
        # False
        return candidate == target

# Example usage:
triplets = [[2,5,3],[1,8,4],[1,7,5]]
target = [2,7,5]
print(mergeTriplets(triplets, target)) # Output: True
```

• [LC.ca](#)

```
class Solution:
    def mergeTriplets(self, triplets: List[List[int]], target:
List[int]) -> bool:
        x, y, z = target
        d = e = f = 0
        for a, b, c in triplets:
            if a <= x and b <= y and c <= z:
                d = max(d, a)
                e = max(e, b)
                f = max(f, c)
        return [d, e, f] == target
```

p.162

p.161

```
object Solution {
  def mergeTriplets(triplets: Array[Array[Int]], target: Array[Int]):
Boolean = {
  val Array(x, y, z) = target
  var (d, e, f) = (0, 0, 0)

  for (Array(a, b, c) <- triplets) {
    if (a <= x && b <= y && c <= z) {
      d = d.max(a)
      e = e.max(b)
      f = f.max(c)
    }
  }

  d == x && e == y && f == z
}
```

p.163

Greedy

#763 Partition Labels

MED

Time Complexity: $O(n)$ where n is the length of the string because we traverse the string twice. Space Complexity: $O(1)$ (or $O(26)$ to be precise) for storing the last occurrences of each character.

Key Insights

- Determine the last occurrence of every character in the string.
- Use a greedy approach to extend the current partition until all characters within it do not appear later in the string.
- When the current index reaches the furthest last occurrence for characters seen so far, a partition can be closed.
- Repeat the process for the remaining section of the string.

Brute Force (Python)

```
# Brute Force  $O(n^2)$  - validate each possible cut point
class Solution:
  def partitionLabels(self, s: str) -> list:
    res, start = [], 0
    for i in range(len(s)):
      seen = set(s[start:i+1])
      if all(s.rfind(c) < i+1 for c in seen):
        res.append(i - start + 1); start = i + 1
    return res
```

Optimal (Python)

p.164

```
# Optimal  $O(n)$  - last occurrence + greedy extend
class Solution:
  def partitionLabels(self, s: str) -> list:
    last = {c: i for i, c in enumerate(s)}
    res = []; start = end = 0
    for i, c in enumerate(s):
      end = max(end, last[c])
      if i == end: res.append(end - start + 1); start = end + 1
    return res
```

Best Answers — All Sites (Python)

• WalkCC

```
class Solution:
  def partitionLabels(self, s: str) -> list[int]:
    ans = []
    letterToRightmostIndex = {c: i for i, c in enumerate(s)}

    l = 0 # the leftmost index of the current running string
    r = 0 # the rightmost index of the current running string

    for i, c in enumerate(s):
      r = max(r, letterToRightmostIndex[c])
      if i == r:
        ans.append(r - l + 1)
        l = r + 1

    return ans
```

• LeetCode

p.165

```
class Solution:
  def partitionLabels(self, s: str) -> List[int]:
    last = {c: i for i, c in enumerate(s)}
    mx = j = 0
    ans = []
    for i, c in enumerate(s):
      mx = max(mx, last[c])
      if mx == i:
        ans.append(i - j + 1)
        j = i + 1
    return ans
```

• SimplyLeet

```
# Python Solution
class Solution:
  def partitionLabels(self, s: str) -> list:
    # Step 1: Record the last occurrence of each character.
    last_occurrence = {char: i for i, char in enumerate(s)}
    # List to store the sizes of partitions.
    partitions = []
    start, end = 0, 0

    # Step 2: Iterate over the string to form partitions.
    for i, char in enumerate(s):
      # Update the end index for the current partition.
      end = max(end, last_occurrence[char])
      # When we reach the end of the partition.
      if i == end:
        partitions.append(end - start + 1)
        start = i + 1 # Move start to the next index.

    return partitions
```

• LC.ca

p.166

```
class Solution:
  def partitionLabels(self, s: str) -> List[int]:
    last = {c: i for i, c in enumerate(s)}
    mx = j = 0
    ans = []
    for i, c in enumerate(s):
      mx = max(mx, last[c])
      if mx == i:
        ans.append(i - j + 1)
        j = i + 1
    return ans
```

Greedy

#678 Valid Parenthesis String

MED

Time Complexity: $O(n)$ where n is the length of the string Space Complexity: $O(1)$

Key Insights

- The '*' character provides flexibility, allowing multiple interpretations.
- A greedy approach can track the range of possible open parentheses count.
- Use two counters to represent the minimum and maximum open count possible at any moment.
- If the maximum count goes negative at any point, the string cannot be valid.
- At the end, the minimum count must be zero for a valid string.

Brute Force (Python)

p.167

p.168

```
# Brute Force O(3^n) - try all replacements of '*'
class Solution:
    def checkValidString(self, s: str) -> bool:
        def valid(s):
            bal = 0
            for c in s:
                if c == '(': bal += 1
                elif c == ')': bal -= 1;
                if bal < 0: return False
            return bal == 0
        def dfs(i, curr):
            if i == len(s): return valid(curr)
            if s[i] != '*': return dfs(i+1, curr+s[i])
            return dfs(i+1, curr+'(') or dfs(i+1, curr+')') or dfs(i+1, curr)
        return dfs(0, '')
```

Optimal (Python)

```
# Optimal O(n) - track range [lo, hi] of open counts
class Solution:
    def checkValidString(self, s: str) -> bool:
        lo = hi = 0
        for c in s:
            if c == '(': lo += 1; hi += 1
            elif c == ')': lo -= 1; hi -= 1
            else: lo -= 1; hi += 1
            if hi < 0: return False
            lo = max(lo, 0)
        return lo == 0
```

Best Answers — All Sites (Python)

• WalkCC

p.169

```
class Solution:
    def checkValidString(self, s: str) -> bool:
        low = 0
        high = 0

        for c in s:
            if c == '(':
                low += 1
                high += 1
            elif c == ')':
                if low > 0:
                    low -= 1
                high -= 1
            else:
                if low > 0:
                    low -= 1
                high += 1
            if high < 0:
                return False

        return low == 0
```

• LeetCode

p.170

```
class Solution:
    def checkValidString(self, s: str) -> bool:
        n = len(s)
        dp = [[False] * n for _ in range(n)]
        for i, c in enumerate(s):
            dp[i][i] = c == '*'
        for i in range(n - 2, -1, -1):
            for j in range(i + 1, n):
                dp[i][j] = (
                    s[i] in '(*' and s[j] in '*)' and (i + 1 == j or
                    dp[i + 1][j - 1])
                ) or any(
                    dp[i][k] and dp[k + 1][j] for k in range(i, j)
                )
        return dp[0][-1]
```

p.171

```
class Solution:
    def checkValidString(self, s: str) -> bool:
        x = 0
        for c in s:
            if c in '(*':
                x += 1
            elif c == ')':
                x -= 1
            else:
                return False
        x = 0
        for c in s[::-1]:
            if c in '*)':
                x += 1
            elif c == '(':
                x -= 1
            else:
                return False
        return True
```

• SimplyLeet

p.172

```
# Python implementation with detailed inline comments

def checkValidString(s):
    # Initialize the minimum and maximum possible count of open
    # parentheses
    minOpen = 0
    maxOpen = 0

    # Iterate over each character in the string
    for char in s:
        if char == '(':
            # For an open parenthesis, both min and max increase by 1
            minOpen += 1
            maxOpen += 1
        elif char == ')':
            # For a close parenthesis, both min and max decrease by 1
            minOpen -= 1
            maxOpen -= 1
        else: # char == '*'
            # '*' can act as a ')' (decrease min) or '(' (increase max)
            minOpen -= 1
            maxOpen += 1
```

p.173

```
# Ensure minOpen does not drop below 0
if minOpen < 0:
    minOpen = 0

# If maxOpen is negative, there are too many ')' to balance
if maxOpen < 0:
    return False

# The string is valid if there is a possible balance where minOpen
# equals 0
return minOpen == 0

# Example usage:
print(checkValidString("(")) # Expected output: True
```

• LC.ca

p.174

```
class Solution:
    def checkValidString(self, s: str) -> bool:
        x = 0
        for c in s:
            if c in '(':
                x += 1
            elif x:
                x -= 1
            else:
                return False
        x = 0
        for c in s[::-1]:
            if c in ')':
                x += 1
            elif x:
                x -= 1
            else:
                return False
        return True
```

p.175

Intervals (Greedy)

NeetCode 150 · Not In 331 · 1 question

p.176

Intervals (Greedy)

#1851 Minimum Interval to Include Each Query

HAR

Time Complexity: $O((N + Q) \log N)$ where N is the number of intervals and Q is the number of queries. Sorting and heap operations dominate the complexity. Space Complexity: $O(N + Q)$ for storing intervals, queries, and the heap.

Key Insights

- Sort the intervals by their start values to efficiently determine when an interval becomes relevant for a given query.
- Process queries in increasing order so that intervals can be added to a data structure (min-heap) as they "start" to cover queries.
- Use a min-heap keyed by the size (length) of the interval to quickly retrieve the smallest interval currently containing the query.
- Remove any intervals from the heap that no longer cover the current query (i.e., their end is less than the query).
- Maintain the original order of queries by storing the indices while sorting.

Brute Force (Python)

p.177

```
# Brute Force  $O(n \cdot q)$  - for each query check all intervals
class Solution:
    def minInterval(self, intervals: list, queries: list) -> list:
        res = []
        for q in queries:
            best = -1
            for l, r in intervals:
                if l <= q <= r:
                    size = r - l + 1
                    if best == -1 or size < best: best = size
            res.append(best)
        return res
```

Optimal (Python)

```
# Optimal  $O((n+q) \log n)$  - sort + min-heap sweep
import heapq
class Solution:
    def minInterval(self, intervals: list, queries: list) -> list:
        intervals.sort()
        indexed_q = sorted(enumerate(queries), key=lambda x: x[1])
        res = [-1] * len(queries); heap = []; i = 0
        for orig_idx, q in indexed_q:
            while i < len(intervals) and intervals[i][0] <= q:
                l, r = intervals[i]; heapq.heappush(heap, (r - l + 1, r)); i += 1
            while heap and heap[0][1] < q: heapq.heappop(heap)
            if heap: res[orig_idx] = heap[0][0]
        return res
```

Best Answers — All Sites (Python)

• [LeetDoocs](#)

p.178

```
class Solution:
    def minInterval(self, intervals: List[List[int]], queries: List[int]) -> List[int]:
        n, m = len(intervals), len(queries)
        intervals.sort()
        queries = sorted((x, i) for i, x in enumerate(queries))
        ans = [-1] * m
        pq = []
        i = 0
        for x, j in queries:
            while i < n and intervals[i][0] <= x:
                a, b = intervals[i]
                heappush(pq, (b - a + 1, b))
                i += 1
            while pq and pq[0][1] < x:
                heappop(pq)
            if pq:
                ans[j] = pq[0][0]
        return ans
```

• [SimplyLeet](#)

p.179

```
import heapq

def minInterval(intervals, queries):
    # Sort intervals by their starting point.
    intervals.sort(key=lambda itv: itv[0])

    # Prepare queries sorted by their value, keeping track of original index.
    sorted_queries = sorted([(q, i) for i, q in enumerate(queries)])

    result = [-1] * len(queries)
    min_heap = []
    i = 0 # for intervals

    # Process each query in ascending order.
    for q, qi in sorted_queries:
        # Add all intervals that start <= current query.
        while i < len(intervals) and intervals[i][0] <= q:
            start, end = intervals[i]
            # Compute the size of the interval.
            size = end - start + 1
            # Push tuple: (size, end) into the min-heap.
            heapq.heappush(min_heap, (size, end))
```

p.180

```
i += 1
# Remove intervals that do not cover the current query.
while min_heap and min_heap[0][1] < q:
    heapq.heappop(min_heap)
# The top of the heap (if any) is the smallest interval
covering the query.
if min_heap:
    result[qi] = min_heap[0][0]
return result

# Example usage
intervals = [[1,4],[2,4],[3,6],[4,4]]
queries = [2,3,4,5]
print(minInterval(intervals, queries)) # Output: [3, 3, 1, 4]
```

• [LC.ca](#)

```
class Solution:
    def minInterval(self, intervals: List[List[int]], queries:
List[int]) -> List[int]:
    n, m = len(intervals), len(queries)
    intervals.sort()
    queries = sorted((x, i) for i, x in enumerate(queries))
    ans = [-1] * m
    pq = []
    i = 0
    for x, j in queries:
        while i < n and intervals[i][0] <= x:
            a, b = intervals[i]
            heappush(pq, (b - a + 1, b))
            i += 1
        while pq and pq[0][1] < x:
            heappop(pq)
        if pq:
            ans[j] = pq[0][0]
    return ans
```

Math & Geometry

NeetCode 150 · Not In 331 · 4 questions

Math & Geometry

#202 Happy Number

EAS

Time Complexity: O(1) - Although the process appears iterative, the sum of squares quickly reduces any number to a bounded range. Space Complexity: O(1) - The number of unique sums computed is bounded, resulting in constant space usage.

Key Insights

- The process involves repeatedly computing the sum of squares of digits.
- If the process reaches 1, the number is happy.
- If the computed values start repeating (cycle detection), the number is not happy.
- The cycle can be detected using a hash set to record previously seen numbers or using a two pointers (Floyd's cycle detection) approach.

Brute Force (Python)

```
# Brute Force - set to detect cycle
class Solution:
    def isHappy(self, n: int) -> bool:
        def digit_sq_sum(x): return sum(int(d)**2 for d in str(x))
        seen = set()
        while n != 1:
            if n in seen: return False
            seen.add(n); n = digit_sq_sum(n)
        return True
```

Optimal (Python)

```
# Optimal O(1) space - Floyd's cycle detection
class Solution:
    def isHappy(self, n: int) -> bool:
        def step(x): return sum(int(d)**2 for d in str(x))
        slow, fast = n, step(n)
        while fast != 1 and slow != fast:
            slow = step(slow); fast = step(step(fast))
        return fast == 1
```

Best Answers — All Sites (Python)

• [WalkCC](#)

```
class Solution:
    def isHappy(self, n: int) -> bool:
        def squaredSum(n: int) -> bool:
            summ = 0
            while n > 0:
                summ += pow(n % 10, 2)
                n //= 10
            return summ

        slow = squaredSum(n)
        fast = squaredSum(squaredSum(n))

        while slow != fast:
            slow = squaredSum(slow)
            fast = squaredSum(squaredSum(fast))

        return slow == 1
```

• [LeetCode](#)

```
class Solution:
    def isHappy(self, n: int) -> bool:
        vis = set()
        while n != 1 and n not in vis:
            vis.add(n)
            x = 0
            while n:
                n, v = divmod(n, 10)
                x += v * v
            n = x
        return n == 1

class Solution:
    def isHappy(self, n: int) -> bool:
        def next(x):
            y = 0
            while x:
                x, v = divmod(x, 10)
                y += v * v
            return y

        slow, fast = n, next(n)
        while slow != fast:
            slow, fast = next(slow), next(next(fast))
        return slow == 1
```

• [SimplyLeet](#)

```
# Function to compute the sum of squares of digits
def sum_of_squares(num):
    total = 0
    while num:
        digit = num % 10 # Extract the last digit
        total += digit * digit # Add its square to the total
        num //= 10 # Remove the last digit
    return total

def isHappy(n):
    seen = set() # Initialize a set to keep track of seen numbers
    # Iterate until n becomes 1 or we detect a cycle
    while n != 1 and n not in seen:
        seen.add(n) # Add current n to the set
        n = sum_of_squares(n) # Compute new n as the sum of squares of
        digits
    return n == 1 # Return True if n is 1, else False

# Example usage:
print(isHappy(19)) # Expected output: True
print(isHappy(2)) # Expected output: False
```

• LC.ca

p.187

```
class Solution:
    def isHappy(self, n: int) -> bool:
        def next(x):
            y = 0
            while x:
                x, v = divmod(x, 10)
                y += v * v
            return y

        slow, fast = n, next(n)
        while slow != fast:
            slow, fast = next(slow), next(next(fast))
        return slow == 1
```

p.188

#66 Plus One

EAS

Time Complexity: $O(n)$ in the worst-case, where n is the number of digits.
Space Complexity: $O(n)$ in the worst-case for the output array (when an extra digit is added).

Key Insights

- The digits are stored from most significant to least significant.
- Start processing from the least significant digit (end of array) to handle potential carry.
- Carefully handle the case where all digits are 9, in which case an additional digit is needed.
- Use an in-place update approach to reduce extra space usage when possible.

Brute Force (Python)

```
# Brute Force – convert to int, add 1, convert back
class Solution:
    def plusOne(self, digits: List) -> List:
        return [int(d) for d in str(int(''.join(map(str, digits))) + 1)]
```

Optimal (Python)

```
# Optimal  $O(n)$  – in-place carry propagation
class Solution:
    def plusOne(self, digits: List) -> List:
        for i in range(len(digits) - 1, -1, -1):
            if digits[i] < 9: digits[i] += 1; return digits
            digits[i] = 0
        return [1] + digits
```

p.189

Best Answers — All Sites (Python)

• WalkCC

```
class Solution:
    def plusOne(self, digits: List[int]) -> List[int]:
        for i, d in reversed(list(enumerate(digits))):
            if d < 9:
                digits[i] += 1
                return digits
            digits[i] = 0
        return [1] + digits
```

• LeetCodecs

```
class Solution:
    def plusOne(self, digits: List[int]) -> List[int]:
        n = len(digits)
        for i in range(n - 1, -1, -1):
            digits[i] += 1
            digits[i] %= 10
            if digits[i] != 0:
                return digits
        return [1] + digits
```

• SimplyLeet

p.190

```
# Function to add one to the array of digits
def plusOne(digits):
    # Process digits starting from the least significant digit
    for i in range(len(digits) - 1, -1, -1):
        # Add one to the current digit
        digits[i] += 1
        # Check if digit is less than 10, no carry needed
        if digits[i] < 10:
            return digits # No further carry, return the result
        # Set current digit to 0 because of carry over
        digits[i] = 0
    # If carry remains after processing all digits, add a new digit at
    # start
    return [1] + digits

# Example usage
# print(plusOne([1, 2, 3])) # Output: [1, 2, 4]
```

• LC.ca

```
class Solution:
    def plusOne(self, digits: List[int]) -> List[int]:
        n = len(digits)
        for i in range(n - 1, -1, -1):
            if digits[i] < 9:
                digits[i] += 1
                return digits

        digits[i] = 0

        return [1] + digits
        # also ok: return [1] + [0]*n
```

#####

```
class Solution(object):
    def plusOne(self, digits):
        """
        :type digits: List[int]
        :rtype: List[int]
        """
        carry = 1

        for i in reversed(range(0, len(digits))):
            digit = (digits[i] + carry) % 10
            carry = 1 if digit < digits[i] else 0
            digits[i] = digit
        if carry == 1:
            return [1] + digits
        return digits
```

p.191

p.192

#43 Multiply Strings

MED

Time Complexity: $O(n * m)$, where n and m are the lengths of `num1` and `num2` respectively. Space Complexity: $O(n + m)$, used for storing the intermediate multiplication results.

Key Insights

- Simulate the multiplication process similar to hand multiplication.
- Use an array to store intermediate results where the index correlation is based on the position of digits.
- Each multiplication may contribute a value that needs to be managed with a carry over in the result array.
- Handle the special case when either input is "0".

Brute Force (Python)

```
# Brute Force - Python's built-in big integers
class Solution:
    def multiply(self, num1: str, num2: str) -> str:
        return str(int(num1) * int(num2))
```

Optimal (Python)

```
# Optimal  $O(m*n)$  - grade school multiplication without big int
class Solution:
    def multiply(self, num1: str, num2: str) -> str:
        m, n = len(num1), len(num2); res = [0] * (m + n)
        for i in range(m - 1, -1, -1):
            for j in range(n - 1, -1, -1):
                mul = int(num1[i]) * int(num2[j]); p1, p2 = i + j, i + j + 1
                total = mul + res[p2]; res[p2] = total % 10; res[p1] += total // 10
            result = ''.join(map(str, res)).rstrip('0')
        return result or '0'
```

Best Answers — All Sites (Python)

• WalkCC

```
class Solution:
    def multiply(self, num1: str, num2: str) -> str:
        s = [0] * (len(num1) + len(num2))

        for i in reversed(range(len(num1))):
            for j in reversed(range(len(num2))):
                mult = int(num1[i]) * int(num2[j])
                summ = mult + s[i + j + 1]
                s[i + j] += summ // 10
                s[i + j + 1] = summ % 10

        for i, c in enumerate(s):
            if c != 0:
                break

        return ''.join(map(str, s[i:]))
```

• LeetDoocs

```
class Solution:
    def multiply(self, num1: str, num2: str) -> str:
        if num1 == "0" or num2 == "0":
            return "0"
        m, n = len(num1), len(num2)
        arr = [0] * (m + n)
        for i in range(m - 1, -1, -1):
            a = int(num1[i])
            for j in range(n - 1, -1, -1):
                b = int(num2[j])
                arr[i + j + 1] += a * b
            for i in range(m + n - 1, 0, -1):
                arr[i - 1] += arr[i] // 10
                arr[i] %= 10
        i = 0
        if arr[0] else 1
        return "".join(str(x) for x in arr[i:])
```

• SimplyLeet

```
# Python implementation with line-by-line comments

def multiply(num1, num2):
    # If either string is "0", the product is "0".
    if num1 == "0" or num2 == "0":
        return "0"

    n, m = len(num1), len(num2)
    # Initialize an array to store the product results.
    result = [0] * (n + m)

    # Multiply each digit of num1 with each digit of num2.
    for i in range(n - 1, -1, -1):
        for j in range(m - 1, -1, -1):
            # Multiply current digits.
            product = int(num1[i]) * int(num2[j])
            # Position in the result array where the product should be
            added.
            pos = i + j + 1
            total = product + result[pos]
            result[pos] = total % 10 # Place the digit at the current
            index.
            result[pos - 1] += total // 10 # Carry over the remaining
            value.

    # Convert the result array into a string, skipping any leading
    zero.
    result_str = ''.join(map(str, result)).rstrip('0')
    return result_str if result_str else "0"

# Example usage:
print(multiply("123", "456")) # Expected output: "56088"
```

• LC.ca

```
class Solution:
    def multiply(self, num1: str, num2: str) -> str:
        if num1 == "0" or num2 == "0":
            return "0"
        m, n = len(num1), len(num2)
        arr = [0] * (m + n)
        for i in range(m - 1, -1, -1):
            a = int(num1[i])
            for j in range(n - 1, -1, -1):
                b = int(num2[j])
                arr[i + j + 1] += a * b
            for i in range(m + n - 1, 0, -1):
                arr[i - 1] += arr[i] // 10
                arr[i] %= 10
        i = 0
        if arr[0] else 1
        return "".join(str(x) for x in arr[i:])
```

#2013 Detect Squares

MED

Time Complexity: • The add operation is $O(1)$. • The count operation is $O(n)$ in the worst-case scenario, where n is the number of distinct points with the same y -coordinate as the query point. Space Complexity: • $O(m)$, where m is the number of added points, since we store counts for each point.

Key Insights

- Leverage the fact that an axis-aligned square has vertical and horizontal edges.
- For a query point (x, y) , iterate over all points that share the same y -axis. For each of these points (px, y) (with $px \neq x$), determine the potential square's side length, $d = |px - x|$.
- Check for the existence of the two other points $(x, y+d)$ and $(px, y+d)$ (or $(x, y-d)$ and $(px, y-d)$) that would complete the square.
- Use a hash map (or dictionary) to store each point along with its count since duplicate points are allowed.

Brute Force (Python)

```
# Brute Force O(n^2) per query – enumerate all point pairs as diagonals
from collections import Counter
class DetectSquares:
    def __init__(self): self.pts = Counter()
    def add(self, point: list) -> None: self.pts[tuple(point)] += 1
    def count(self, point: list) -> int:
        px, py = point; res = 0
        for (x1, y1), c1 in self.pts.items():
            for (x2, y2), c2 in self.pts.items():
                if abs(x1-px) == abs(y1-py) and abs(x1-px) > 0:
                    if x2 == x1 and y2 == py:
                        res += c1 * c2 * self.pts.get((px, y1), 0)
        return res
```

Optimal (Python)

```
# Optimal O(n) per query – enumerate diagonal x-coordinate
from collections import Counter, defaultdict
class DetectSquares:
    def __init__(self): self.pts = Counter(); self.x_set = defaultdict(set)
    def add(self, point: list) -> None:
        x, y = point; self.pts[(x, y)] += 1; self.x_set[x].add(y)
    def count(self, point: list) -> int:
        px, py = point; res = 0
        for x3 in self.x_set:
            side = abs(x3 - px)
            if side == 0: continue
            for dy in (side, -side):
                res += (self.pts[(x3, py+dy)] * self.pts[(x3, py-dy)] *
self.pts[(px, py+dy)])
        return res
```

Best Answers — All Sites (Python)

p.199

• WalkCC

```
class DetectSquares:
    def __init__(self):
        self.pointCount = collections.Counter()

    def add(self, point: list[int]) -> None:
        self.pointCount[tuple(point)] += 1

    def count(self, point: list[int]) -> int:
        x1, y1 = point
        ans = 0
        for (x3, y3), c in self.pointCount.items():
            if x1 != x3 and abs(x1 - x3) == abs(y1 - y3):
                ans += c * self.pointCount[(x1, y3)] * self.pointCount[(x3,
y1)]
        return ans
```

• LeetCode

```
class DetectSquares:
    def __init__(self):
        self.cnt = defaultdict(Counter)

    def add(self, point: List[int]) -> None:
        x, y = point
        self.cnt[x][y] += 1

    def count(self, point: List[int]) -> int:
        x1, y1 = point
        if x1 not in self.cnt:
            return 0
        ans = 0
        for x2 in self.cnt.keys():
            if x2 != x1:
                d = x2 - x1
                ans += self.cnt[x2][y1] * self.cnt[x1][y1 + d] *
self.cnt[x2][y1 + d]
                ans += self.cnt[x2][y1] * self.cnt[x1][y1 - d] *
self.cnt[x2][y1 - d]
        return ans

# Your DetectSquares object will be instantiated and called as such:

# obj = DetectSquares()
# obj.add(point)
# param_2 = obj.count(point)
```

• SimplyLeet

p.201

```
# Python implementation of DetectSquares
from collections import Counter

class DetectSquares:
    def __init__(self):
        # Use a counter to store the frequency of each point (x, y)
        self.point_counts = Counter()

    def add(self, point):
        x, y = point[0], point[1]
        # Increment the count for the point (x, y)
        self.point_counts[(x, y)] += 1

    def count(self, point):
        x, y = point[0], point[1]
        total_squares = 0
        # Iterate through all points in our data structure
        for (px, py), count in self.point_counts.items():
            # Only consider points that share the same y-coordinate
            # with the query point
            if py == y and px != x:
                # Calculate the potential square's side length
                d = px - x
```

p.202

```
# Check for a square above the query point
total_squares += count * self.point_counts.get((x, y +
d), 0) * self.point_counts.get((px, y + d), 0)
# Check for a square below the query point
total_squares += count * self.point_counts.get((x, y -
d), 0) * self.point_counts.get((px, y - d), 0)
return total_squares

# Example Usage:
# detectSquares = DetectSquares()
# detectSquares.add([3, 10])
# detectSquares.add([11, 2])
# detectSquares.add([3, 2])
# print(detectSquares.count([11, 10])) # Output: 1
```

• LC.ca

```
class DetectSquares:
    def __init__(self):
        self.cnt = defaultdict(Counter)

    def add(self, point: List[int]) -> None:
        x, y = point
        self.cnt[x][y] += 1

    def count(self, point: List[int]) -> int:
        x1, y1 = point
        if x1 not in self.cnt:
            return 0
        ans = 0
        for x2 in self.cnt.keys():
            if x2 != x1:
                d = x2 - x1
                ans += self.cnt[x2][y1] * self.cnt[x1][y1 + d] *
self.cnt[x2][y1 + d]
                ans += self.cnt[x2][y1] * self.cnt[x1][y1 - d] *
self.cnt[x2][y1 - d]
        return ans

# Your DetectSquares object will be instantiated and called as such:

# obj = DetectSquares()
# obj.add(point)
# param_2 = obj.count(point)
```

p.203

p.204

Bit Manipulation

NeetCode 150 · Not In 331 · 1 question

p.205

Bit Manipulation

#371 Sum of Two Integers

MED

Time Complexity: O(1) – The loop runs a fixed number of times based on the number of bits. Space Complexity: O(1) – Only constant extra space is used.

Key Insights

- Use bitwise XOR (^) to perform addition without carrying.
- Use bitwise AND (&) followed by a left shift (<<) to calculate the carry.
- Repeat the process until there is no carry.
- This method simulates the addition process using binary arithmetic.

Brute Force (Python)

```
# Full adder simulation with 32-bit masking
class Solution:
    def getSum(self, a: int, b: int) -> int:
        MASK = 0xFFFFFFFF; MAX = 0x7FFFFFFF
        while b & MASK:
            carry = (a & b) << 1; a = a ^ b; b = carry
        return a if a <= MAX else ~(a ^ MASK)
```

Optimal (Python)

```
# Optimal – cleaner 32-bit masking throughout
class Solution:
    def getSum(self, a: int, b: int) -> int:
        MASK = 0xFFFFFFFF; MAX = 0x7FFFFFFF
        while b:
            carry = ((a & b) << 1) & MASK
            a = (a ^ b) & MASK; b = carry
        return a if a <= MAX else ~(a ^ MASK)
```

p.206

Best Answers — All Sites (Python)

• WalkCC

```
class Solution:
    def getSum(self, a: int, b: int) -> int:
        mask = 0xFFFFFFFF
        MAX = 2000

        while b != 0:
            a, b = (a ^ b) & mask, ((a & b) << 1) & mask

        return a if a < MAX else ~(a ^ mask)
```

• LeetCode

```
class Solution:
    def getSum(self, a: int, b: int) -> int:
        a, b = a & 0xFFFFFFFF, b & 0xFFFFFFFF
        while b:
            carry = ((a & b) << 1) & 0xFFFFFFFF
            a, b = a ^ b, carry
        return a if a < 0x80000000 else ~(a ^ 0xFFFFFFFF)
```

• SimplyLeet

p.207

```
# Python implementation to add two integers without using + or -
def getSum(a, b):
    # Continue till there is no carry
    while b != 0:
        # XOR gets the sum excluding carries
        sum_without_carry = a ^ b
        # AND finds carries, left shift to apply them to the next bit
        carry = (a & b) << 1
        # Update a with sum without carry and b with carry
        a = sum_without_carry
        b = carry
    return a

# Example usage:
result = getSum(1, 2)
print("Sum:", result) # Output: Sum: 3
```

p.208

Quick Review

Key insights + complexity at a glance · all 32 questions

p.209

p.210

#567 Permutation in String

MED

Key Insights

- Use a sliding window approach to examine substrings of s2 of length equal to s1.
- Maintain frequency counts of characters for both s1 and the current window in s2.
- Compare the frequency counts to determine if a valid permutation exists.
- Increment the window by adding a new character and removing the leftmost one, keeping the operation efficient.

Space & Time

Time Complexity: O(n) where n is the length of s2 (each character is processed a constant number of times).

Space Complexity: O(1) since the frequency count arrays have a fixed size (26 for lowercase English letters).

Solution

The solution uses two fixed-length arrays (of size 26) to store character frequencies for s1 and the current window in s2. First, initialize the frequency arrays by processing s1 and the first window of s2. Then, slide the window one character at a time, updating the counts by incrementing the count for the new character and decrementing the count for the character that is no longer in the window. After each move, compare both frequency arrays; if they match, s2 contains a permutation of s1. This technique avoids repeatedly computing the frequency distribution from scratch, making the algorithm efficient.

#563 Car Fleet

MED

Key Insights

- Since cars cannot pass each other, the relative order (by starting position) is maintained.
- Sorting the cars based on their starting positions (descending order) helps to simulate the catching-up process.
- Computing the time each car takes to reach the target is critical.
- Compare the current car's time to the time of the fleet ahead: if it takes longer, it forms a new fleet; otherwise, it gets merged.
- A stack-like approach (tracking the slowest arrival time seen so far) reduces the problem to a single pass after sorting.

Space & Time

Time Complexity: O(n log n) due to the sorting step.

Space Complexity: O(n) for storing the computed times and sorted car information.

Solution

The solution involves sorting the cars by their starting positions in descending order so that we process the cars from nearest to the target to farthest. For each car, calculate the time to reach the target using the formula (target - position) / speed. As we iterate:

- If a car's time is greater than the slowest time encountered (tracked by a variable or stack), it cannot catch up to the fleet ahead and forms a new fleet.
- Otherwise, it merges into the current fleet. This approach ensures that we count each distinct fleet accurately with a single pass through the sorted list.

#875 Koko Eating Bananas

MED

Key Insights

- The required eating speed k lies in the range [1, max(piles)].
- For a given k, one can calculate the total hours needed by summing up ceil(pile / k) for each pile.
- The problem can be solved efficiently using binary search to find the minimum k that satisfies the time constraint.
- Early termination in the hour calculation can be used if the computed hours exceed h.

Space & Time

Time Complexity: O(n log(max(pile))) where n is the number of piles and max(pile) is the maximum bananas in a pile.

Space Complexity: O(1) extra space beyond the input.

Solution

We use binary search on the candidate eating rate k. The search space starts at 1 (minimum possible speed) and goes up to the maximum number of bananas in any pile. For each candidate k, we simulate the number of hours Koko would take by calculating hours += (pile + k - 1) // k for each pile (this formula performs the ceiling division). If the total hours are less than or equal to h, then k might be a valid solution and we try to lower it; otherwise, we increase k. The binary search continues until we converge on the minimum valid eating speed.

#138 Copy List with Random Pointer

MED

Key Insights

- Use a hash map (or dictionary) to map each original node to its corresponding copy.
- Perform two passes: the first to create a copy of each node, and the second to assign the next and random pointers using the hash map.
- An alternative interleaving approach exists but using a hash table is typically more straightforward.

Space & Time

Time Complexity: O(n) - Each node is processed a constant number of times.

Space Complexity: O(n) - A hash map is used to store the mapping from original nodes to their copies.

Solution

The solution uses a two-pass algorithm. In the first pass, we iterate through the original list and for each node, create a new node (deep copy) while saving the mapping between the original and copy in a hash table. In the second pass, we iterate over the original list again to assign the next and random pointers of the new nodes using the previously built map. This guarantees that every pointer in the copied list corresponds to a new node and not the original.

#1448 Count Good Nodes in Binary Tree

MED

Key Insights

- Perform a tree traversal (DFS or BFS) from the root.
- Keep track of the maximum value encountered along the current path.
- A node is "good" if its value is greater than or equal to the maximum value seen so far.
- Update the maximum value as you traverse deeper in the tree.
- The root is always a good node.

Space & Time

Time Complexity: O(n), where n is the number of nodes, since each node is visited exactly once.

Space Complexity: O(h), where h is the height of the tree (O(n) in the worst-case of a skewed tree).

Solution

We can solve this problem using Depth-First Search (DFS) recursion. Starting from the root, we pass along the current maximum value encountered on the path. For each node, we check if its value is greater than or equal to this maximum. If it is, we count it as a good node and update the maximum value if necessary. This process is continued recursively for both the left and right children of the node. Finally, we sum the counts from both subtrees.

#703 Kth Largest Element in a Stream

EAS

Key Insights

- Use a min-heap (priority queue) of fixed size k.
- The heap always contains the k largest elements; the smallest element in the heap is the kth largest overall.
- When a new test score is added:
- If the heap has less than k elements, simply add the score.
- If the heap is full and the new score is greater than the smallest (heap root), replace the root.
- Each add operation runs in O(log k) time.

Space & Time

Time Complexity: O(log k) per add operation, O(n log k) to build the initial heap.

Space Complexity: O(k) for maintaining the heap.

Solution

We use a min-heap to track the k largest numbers. When initializing, we add numbers from the given list to the heap, ensuring not to exceed a size of k. For each add operation, if the new number is greater than the heap minimum, we replace the minimum. The kth largest number is always accessible as the minimum element in the heap.

Key Insights

- Use a max-heap (or simulate one) to always access the two largest stones quickly.
- If the two stones are equal, both are removed; otherwise, the difference is pushed back into the heap.
- Continue the process until the heap has one or no stone left.
- For languages without a max-heap in their standard library (e.g., Python), simulate one by pushing negative values.

Space & Time

Time Complexity: $O(n \log n)$ – Each stone removal and insertion takes $O(\log n)$ and in the worst-case scenario, we perform $O(n)$ operations.

Space Complexity: $O(n)$ – The heap stores all the stones initially.

Solution

We solve the problem by using a priority queue (max-heap). In Python, we simulate a max-heap by inserting the negative of each stone's weight into a min-heap. In JavaScript, due to the lack of a built-in heap structure, we sort the array in descending order at each iteration (acceptable given the small constraints). In C++ and Java, we use the standard priority_queue. The algorithm repeatedly extracts the two largest stones, computes their difference, and if the difference is non-zero, it is inserted back into the heap. This process repeats until there is at most one stone left.

p.217

Key Insights

- Use a global timestamp to order tweets as they are posted.
- Maintain a mapping of each user to the list of tweets they posted.
- Maintain a mapping of each user to the set of users they follow.
- For getNewsFeed, merge tweets from the user and their followees using a heap to efficiently retrieve the 10 most recent tweets.
- Ensure that a user always follows themselves to simplify retrieving their own tweets.

Space & Time

Time Complexity:

- postTweet: $O(1)$
- follow / unfollow: $O(1)$
- getNewsFeed: $O(N \log K)$ where N is the number of followees and K is the average number of tweets considered (capped at 10 tweets per merge iteration)

Space Complexity:

$O(U + T)$ where U is the number of users (for follow mappings) and T is the total number of tweets stored.

Solution

We use two main data structures: a hash table to store tweets for each user and another hash table for follow relationships. Each tweet is stored as a tuple of (timestamp, tweetId). A global timestamp counter provides a unique ordering. A min-heap (or max-heap using negative timestamp values) is used to merge the tweets from different users efficiently. When retrieving the news feed, we look up the tweet lists for the given user's followees (including self) and use the heap to extract the 10 most recent tweets.

p.218

Key Insights

- Sort the candidates first to easily skip duplicates.
- Use backtracking to explore all possible combinations.
- Avoid selecting the same number (at the same recursive level) to avoid duplicate combinations.
- Terminate recursion when the running sum exceeds the target.
- Use indices rather than reusing elements to ensure each number is used at most once.

Space & Time

Time Complexity: $O(2^n * k)$ in the worst case, where n is the number of candidates and k is the average length of a combination.

Space Complexity: $O(k)$ for the recursion stack and current combination (not including the space occupied by the output).

Solution

The solution is based on a backtracking algorithm. First, the candidates are sorted to handle duplicates easily. Then, a recursive function is used to build combinations. At each recursion level, iterate through the candidates starting from the current index. For each candidate:

- Skip the candidate if it is the same as the previous one (to avoid duplicates).
- If the candidate can be added without exceeding the target, add it to the current combination.
- Recurse with the next index (ensuring each candidate is used only once).
- Backtrack by removing the candidate from the current combination once completed.

Data structures used include:

- An array or list to hold the current combination.
- A results list to store all valid combinations.
- Sorting is leveraged to simplify duplicate elimination.

p.219

Key Insights

- Use backtracking to explore all possible ways to partition the string.
- At each step, check whether the current substring is a palindrome.
- If it is, include that substring in the current partition and recursively partition the rest of the string.
- The limit on the string length (at most 16 characters) allows an exponential backtracking approach.
- Optimizations like caching palindrome checks can be used, though the simple approach is typically sufficient.

Space & Time

Time Complexity: $O(n * 2^n)$ in the worst case, due to exploring all possible partitions and checking palindromes.

Space Complexity: $O(n)$ for recursion stack and temporary storage for current partition.

Solution

The solution uses the backtracking algorithm:

- Define a helper function that will recursively try to partition the string.
- For each possible split, check if the substring is a palindrome.
- If it is, add the substring to the current partition list and recursively call the helper on the remaining substring.
- When the end of the string is reached, add the current partition list to the final result.
- Backtrack by removing the last added substring and try the next possible partition. This method systematically explores every possible palindrome partitioning of the string.

p.220

Key Insights

- The problem can be solved using a Union Find (Disjoint Set Union) data structure.
- As we iterate over each edge, we check whether the two vertices are already connected.
- If connecting the two vertices would create a cycle (i.e., they already belong to the same set), then the current edge is redundant.
- Alternatives like DFS or BFS are possible, but Union Find offers a more straightforward and efficient solution.

Space & Time

Time Complexity: $O(n * \alpha(n))$ where $\alpha(n)$ is the inverse Ackermann function (almost constant in practice).

Space Complexity: $O(n)$, for storing the parent array used in the Union Find structure.

Solution

We use the Union Find algorithm by maintaining an array where each node points to its parent. For each edge, we perform a find operation on both endpoints:

- If the endpoints belong to the same set, adding this edge would create a cycle and it is marked as redundant.
- Otherwise, we union the two sets. This algorithm efficiently detects the extra edge that introduces a cycle in the graph.

Key Insights

- This is a single-source shortest path problem on a directed graph.
- Since all edge weights are non-negative, Dijkstra's algorithm is well-suited for this problem.
- Construct an adjacency list for the graph.
- Utilize a min-heap (priority queue) to efficiently select the next node with the smallest cumulative travel time.
- The answer is the maximum travel time among all reachable nodes; if any node is unreachable, return -1.

Space & Time

Time Complexity: $O(E \log V)$ where E is the number of edges and V is the number of nodes.

Space Complexity: $O(V + E)$ due to the adjacency list and distance storage.

Solution

We solve the problem using Dijkstra's algorithm. First, an adjacency list is built using the input times array. A distance map (or array) records the shortest known distance from the starting node k to all other nodes, initialized to infinity except for k which is set to 0. A min-heap stores nodes prioritized by their current shortest distance.

The algorithm repeatedly extracts the node with the smallest distance from the heap. For each neighbor of the current node, it checks if a shorter path is found via the current node; if so, the neighbor's distance is updated and it is added to the heap. Once all nodes are processed, the answer is the maximum distance from the k node. If any node remains unreachable (has distance equal to infinity), return -1.

p.222

p.221

Graphs — Advanced #332 Reconstruct Itinerary HAR Key Insights - Model the tickets as a directed graph where airports are nodes. - The problem is akin to finding an Eulerian path through the graph. - Use Depth-First Search (DFS) with Hierholzer's algorithm to construct the itinerary. - To ensure lexical order, sort the destination lists in reverse order so that the smallest lexical destination can be popped during DFS. - Build the itinerary in reverse during backtracking and then reverse it to get the final order. Space & Time Time Complexity: $O(E \log E)$, where E is the number of tickets, due to the sorting of edges. Space Complexity: $O(E)$, to store the graph and recursion stack. Solution Construct an adjacency list (graph) mapping each departure airport to a list of destination airports sorted in reverse lexicographical order. Perform DFS starting from "JFK". During DFS, traverse while there are destinations available from the current airport. Append airports to the itinerary as you backtrack. Reverse the built itinerary at the end to obtain the correct order. p.223	Graphs — Advanced #1584 Min Cost to Connect All Points MED Key Insights - The problem can be reinterpreted as finding a Minimum Spanning Tree (MST) on a complete graph. - Manhattan distance is used to calculate the weight/cost between every pair of vertices. - Prim's or Kruskal's algorithm can be applied to solve the MST problem. - A direct construction of the complete graph is unnecessary; distances can be computed on the fly, which is efficient given the problem constraints. Space & Time Time Complexity: $O(n^2)$ Space Complexity: $O(n)$ Solution We can solve the problem using Prim's algorithm. We start from an arbitrary point (for example, the first point) and iteratively add the point that has the smallest distance to the growing MST. Instead of explicitly constructing all the edges, we compute the Manhattan distance on the fly between the current MST and the remaining points. An array is used to track the minimum connection cost for each point that is not yet included in the tree. At each iteration, we select the point with the smallest cost to add into the MST and update the costs for the remaining points. This approach guarantees that we connect all points with the minimum total cost. p.224	Graphs — Advanced #778 Swim in Rising Water HAR Key Insights - The problem can be viewed as finding a path from the top-left to bottom-right where the cost is the maximum elevation encountered on that path. - A greedy approach using Dijkstra's algorithm (or a min-heap based BFS) ensures that we always consider the next move with the smallest current elevation requirement. - Once the destination is reached, the maximum elevation along that path is the minimum time required. - Alternative approaches include binary search on time combined with a DFS/BFS check to see if a valid path exists. Space & Time Time Complexity: $O(n^2 \log(n^2))$ which simplifies to $O(n^2 \log n)$ since each cell is processed once and we use a priority queue. Space Complexity: $O(n^2)$ due to the visited matrix and the priority queue. Solution We can solve the problem using a min-heap based BFS (Dijkstra's algorithm). Start at cell $(0, 0)$ with its elevation as the initial cost, and use a min-heap to always expand the cell with the lowest required water level. For each popped cell, update the maximum elevation encountered on the current path. When reaching the target cell $(n - 1, n - 1)$, this value represents the least time needed. We maintain a visited matrix to avoid reprocessing cells. p.225
Dynamic Programming 1D #746 Min Cost Climbing Stairs EAS Key Insights - Use Dynamic Programming to solve overlapping subproblems. - The minimum cost to reach a step is the cost of that step plus the minimum cost of reaching one of the two previous steps. - The final result is the minimum cost to reach the last or second last step since you can jump from either directly to the top. Space & Time Time Complexity: $O(n)$ Space Complexity: $O(n)$ (can be optimized to $O(1)$ by using only two variables) Solution We use a dynamic programming approach. Define a dp array where $dp[i]$ is the minimum cost required to reach step i. Initialize $dp[0]$ with $cost[0]$ and $dp[1]$ with $cost[1]$ because you can start at either step. For each subsequent step, the recurrence is $dp[i] = cost[i] + \min(dp[i-1], dp[i-2])$. Finally, the answer is the minimum of $dp[n-1]$ and $dp[n-2]$ since you can reach the top from either of these steps. This method efficiently computes the answer in one pass through the array. p.226	Dynamic Programming 1D #213 House Robber II MED Key Insights - The houses are arranged in a circle, meaning the first and last houses are neighbors. - Because of the circular arrangement, you cannot rob both the first and last houses. - The problem can be reduced into two separate linear House Robber problems: - One considering houses from index 1 to $n-1$ (exclude the first house). - The other considering houses from index 0 to $n-2$ (exclude the last house). - Use dynamic programming with constant space (using two variables) to compute the solution for a linear array of houses. Space & Time Time Complexity: $O(n)$, where n is the number of houses. Space Complexity: $O(1)$, as only a constant amount of extra space is used. Solution To handle the circular dependency, we solve two linear versions of the problem: - Exclude the first house and calculate the maximum rob amount for houses from index 1 to $n-1$. - Exclude the last house and calculate the maximum rob amount for houses from index 0 to $n-2$. Then, the final answer is the maximum of these two calculated values. We use dynamic programming by iterating through the list and at each step, calculating the maximum money that can be robbed up to that point with two variables. p.227	Dynamic Programming 1D #647 Palindromic Substrings MED Key Insights - Every individual character is a palindrome. - Palindromes can be expanded from a center point; centers can be single characters (odd length) or between two characters (even length). - The "expand around center" technique efficiently counts palindromes in $O(n^2)$ time. - Dynamic programming could also be used but requires extra space and is more complex to implement. Space & Time Time Complexity: $O(n^2)$ where n is the length of the string. Space Complexity: $O(1)$ if using the expand-around-center method (ignoring recursion/stack overhead). Solution The solution employs the expand-around-center approach: - Iterate over each index of the string considering it as the center of a palindrome. - For each center, expand in both directions (left and right) as long as the characters are the same and within bounds. - Count every valid palindrome found during the expansion. - Repeat for both odd-length centers (a single index) and even-length centers (between two indices). This method efficiently checks all potential palindromic substrings without extra space for dynamic programming tables. p.228

Key Insights

- Use dynamic programming to build up the number of ways to form every amount up to the target.
- A one-dimensional dp array suffices, where dp[j] represents the number of ways to form sum j.
- When processing each coin, iterate over amounts from the coin value to the target, updating dp.
- Order of iteration ensures that combinations, not permutations, are counted.

Space & Time

Time Complexity: $O(n \times \text{amount})$, where n is the number of coins.

Space Complexity: $O(\text{amount})$, due to the dp array used for computations.

Solution

We solve the problem by using dynamic programming with a one-dimensional dp array. The dp array is sized as amount + 1 with dp[0] initialized to 1, representing the single way to achieve the amount of 0 (by choosing no coins). For each coin in the coins array, we update every dp[j] for j from the coin's value up to the target amount: dp[j] is increased by dp[j - coin]. This ensures we consider each coin only once per combination, hence avoiding duplicate counts of the same combination in different orders.

Key Insights

- The problem can be converted into a subset sum problem by considering two groups: numbers with a '+' sign and numbers with a '-' sign.
- Let the sum of numbers with a '+' sign be sum(P) and with a '-' sign be sum(N). Then the problem reduces to solving:
 - $\text{sum}(P) - \text{sum}(N) = \text{target}$
 - and $\text{sum}(P) + \text{sum}(N) = \text{totalSum}$, where totalSum is the sum of all numbers in nums.
- From these equations, we derive $\text{sum}(P) = (\text{target} + \text{totalSum}) / 2$.
- We only proceed if (target + totalSum) is even and not negative; otherwise, there are no valid expressions.
- Dynamic programming is used to count the number of subsets that sum to sum(P).

Space & Time

Time Complexity: $O(n \times \text{subsetSum})$, where n is the length of the nums array and subsetSum = (target + totalSum)/2.

Space Complexity: $O(\text{subsetSum})$, as we use a one-dimensional DP array.

Solution

The solution converts the problem into a subset sum DP problem. First, compute the total sum of the array. If (target + totalSum) is not even or target is greater than totalSum, return 0 since no valid configuration exists. Otherwise, compute the required subset sum (subsetSum) as (target + totalSum) / 2 and use a dynamic programming array to count the number of ways to reach subsetSum. The DP approach iterates through each number in the array, updating the counts using a reverse iteration to ensure each number is only used once.

Key Insights

- The problem asks if s3 can be constructed by preserving the order of characters in s1 and s2.
- A dynamic programming approach is effective by defining dp[i][j] to represent whether s3[0 ... i+j-1] is an interleaving of s1[0 ... i-1] and s2[0 ... j-1].
- The solution must check that the total lengths match; otherwise, it is immediately false.
- Transitions are based on whether the current character in s3 matches the next char in s1 or s2.
- Space optimization to $O(s2.length)$ is possible by reusing a 1D DP array.

Space & Time

Time Complexity: $O(n \times m)$ where n = length of s1 and m = length of s2.

Space Complexity: $O(n \times m)$ with the basic DP solution. (Can be optimized to $O(m)$ with further space reduction.)

Solution

We use a 2D dynamic programming table where dp[i][j] is true if s3[0 ... i+j-1] can be formed by interleaving s1[0 ... i-1] and s2[0 ... j-1]. First, we check if the lengths of s1 and s2 add up to s3's length; if not, return false. Then, we initialize dp[0][0] to true and fill in the first row and column based on whether the characters in s2 or s1 correspond to s3. For the rest of the table, dp[i][j] is true if either the previous state dp[i-1][j] is true and s1[i-1] matches s3[i+j-1], or dp[i][j-1] is true and s2[j-1] matches s3[i+j-1]. The answer is found in dp[s1.length][s2.length].

Key Insights

- Utilize a greedy algorithm to make optimal jumps.
- Track the furthest index that can be reached within the current jump range.
- When the current index reaches the end of the current range, a jump is made and the range updates to the furthest reachable index.
- Edge case: if the array has only one element, no jumps are needed.

Space & Time

Time Complexity: $O(n)$ - Single pass through the array.

Space Complexity: $O(1)$ - Only a few variables are used for tracking jumps and ranges.

Solution

We use a greedy approach by iterating through the array and keeping track of:

- The furthest index reachable from the current range (furthest).
- The end of the current jump's range (currentEnd).
- The number of jumps made (jumps).

At each iteration, we update the furthest reachable index. When the current index reaches the currentEnd, it indicates that one jump has been used and we update currentEnd to furthest. This ensures we are always making the optimal jump to cover the longest distance possible until the end of the array is reached.

Key Insights

- The total number of cards must be divisible by groupSize.
- Use a frequency count (hash table or map) to record the occurrence of each card.
- Always start forming groups from the smallest available card to maintain the consecutive order.
- For each card, if its count is non-zero, check that the subsequent groupSize-1 cards exist with at least as many counts.
- Decrease the counts accordingly for each valid consecutive group.

Space & Time

Time Complexity: $O(n \log n)$ due to sorting the unique keys from the hand.

Space Complexity: $O(n)$ for storing the frequency count of the cards.

Solution

The solution uses a greedy approach along with a frequency hash map. The main steps are:

- Verify that the total number of cards is divisible by groupSize.
- Count the frequency of each card.
- Sort the keys of the frequency map.
- Iterate over the sorted keys and for each card with a positive count, try to form a group of groupSize consecutive cards.
- For each card in the group, decrease the frequency by the number of groups formed.
- If at any point the required card is missing or its count is insufficient, return false.
- If all groups can be formed successfully, return true.

Key Insights

- Only consider triplets that are "valid" in the sense that none of its values exceed the corresponding target value.
- For each component of the target, there must be at least one valid triplet that provides that maximum value.
- By taking the element-wise maximum across all valid triplets, if we can recreate the target, then the target can be formed using a series of allowed operations.

Space & Time

Time Complexity: $O(n)$, where n is the number of triplets.

Space Complexity: $O(1)$ extra space.

Solution

The solution approach involves iterating over the list of triplets and only considering those triplets that do not exceed the target values in any position. For each valid triplet, update a candidate triplet by taking the element-wise maximum with the current candidate. Finally, check if the candidate exactly matches the target triplet. The algorithm uses simple array operations and avoids any extra space beyond a few variables.

#763 Partition Labels

MED

Key Insights

- Determine the last occurrence of every character in the string.
- Use a greedy approach to extend the current partition until all characters within it do not appear later in the string.
- When the current index reaches the furthest last occurrence for characters seen so far, a partition can be closed.
- Repeat the process for the remaining section of the string.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string because we traverse the string twice.
Space Complexity: $O(1)$ (or $O(26)$ to be precise) for storing the last occurrences of each character.

Solution

We first iterate through the string to store the last index at which each character appears. Then, we traverse the string while maintaining two pointers: one for the start of the current partition and one to track the furthest last occurrence (end) of any character seen so far. For each character, we update the end pointer to be the maximum of its current value and the last index of that character. When the current index equals the end pointer, it means that all characters in the current partition do not appear later. We then determine the size of this partition, add it to the result list, and move to the next partition. We repeat this until the entire string is processed.

p.235

Data structures used: Sorting arrays and a min-heap for efficient retrieval and removal.

p.238

#678 Valid Parenthesis String

MED

Key Insights

- The `'s'` character provides flexibility, allowing multiple interpretations.
- A greedy approach can track the range of possible open parentheses count.
- Use two counters to represent the minimum and maximum open count possible at any moment.
- If the maximum count goes negative at any point, the string cannot be valid.
- At the end, the minimum count must be zero for a valid string.

Space & Time

Time Complexity: $O(n)$ where n is the length of the string
Space Complexity: $O(1)$

Solution

We use a greedy algorithm by maintaining two counters:
– `minOpen` – the minimum number of open parentheses that could be present. – `maxOpen` – the maximum number of open parentheses that could be present.
For every character in the string:
– For `'('`, increment both `minOpen` and `maxOpen`. – For `)'`, decrement both `minOpen` and `maxOpen`. – For `'s'`, decrement `minOpen` (treat as `)'`) and increment `maxOpen` (treat as `'('`).
Clamp `minOpen` to not go below 0, since negative count of open parentheses is not possible in any valid interpretation. If `maxOpen` drops below 0 at any point, it means there are more closing brackets than can be balanced by available `'s'` and `'('`, so the string is invalid. Finally, if `minOpen` is 0, the string can be balanced, otherwise it is invalid.

p.236

Math & Geometry

#202 Happy Number

EAS

Key Insights

- The process involves repeatedly computing the sum of squares of digits.
- If the process reaches 1, the number is happy.
- If the computed values start repeating (cycle detection), the number is not happy.
- The cycle can be detected using a hash set to record previously seen numbers or using a two pointers (Floyd's cycle detection) approach.

Space & Time

Time Complexity: $O(1)$ – Although the process appears iterative, the sum of squares quickly reduces any number to a bounded range.
Space Complexity: $O(1)$ – The number of unique sums computed is bounded, resulting in constant space usage.

Solution

The solution involves computing the sum of the squares of a number's digits in each iteration. We use a hash set to track numbers that have already been seen. If a number repeats, it means we are in a cycle and the number is not happy. If the number becomes 1, it is a happy number. Data structures used include a hash set for cycle detection, and the algorithm uses iterative computation.

p.239

#1851 Minimum Interval to Include Each Query

HAR

Key Insights

- Sort the intervals by their start values to efficiently determine when an interval becomes relevant for a given query.
- Process queries in increasing order so that intervals can be added to a data structure (min-heap) as they "start" to cover queries.
- Use a min-heap keyed by the size (length) of the interval to quickly retrieve the smallest interval currently containing the query.
- Remove any intervals from the heap that no longer cover the current query (i.e., their end is less than the query).
- Maintain the original order of queries by storing the indices while sorting.

Space & Time

Time Complexity: $O((N + Q) \log N)$ where N is the number of intervals and Q is the number of queries. Sorting and heap operations dominate the complexity.
Space Complexity: $O(N + Q)$ for storing intervals, queries, and the heap.

Solution

The solution uses a two-step approach:
– Sort the queries while keeping track of their original indices. – Sort intervals by their starting values. For each query (in increasing order), add intervals that start before or at this query to a min-heap, where each entry is the size of the interval along with its end value. Then, remove intervals from the heap that do not cover the query (their end value is less than the query). The heap's top element (if available) holds the smallest interval size that contains the query. – Place the answer in a result array at the position corresponding to the original query order.

p.237

Math & Geometry

#66 Plus One

EAS

Key Insights

- The digits are stored from most significant to least significant.
- Start processing from the least significant digit (end of array) to handle potential carry.
- Carefully handle the case where all digits are 9, in which case an additional digit is needed.
- Use an in-place update approach to reduce extra space usage when possible.

Space & Time

Time Complexity: $O(n)$ in the worst-case, where n is the number of digits.
Space Complexity: $O(n)$ in the worst-case for the output array (when an extra digit is added).

Solution

We iterate from the end of the digits array towards the beginning, adding one to the current digit. If the digit becomes 10, we set it to 0 and propagate the carry to the next digit. If no carry is needed, we can return the result immediately. If we finish processing the array and still have a carry (e.g., the number was all 9s), we create a new array with one extra digit at the beginning set to 1.

p.240

#43 Multiply Strings

MED

Key Insights

- Simulate the multiplication process similar to hand multiplication.
- Use an array to store intermediate results where the index correlation is based on the position of digits.
- Each multiplication may contribute a value that needs to be managed with a carry over in the result array.
- Handle the special case when either input is "0".

Space & Time

Time Complexity: $O(n * m)$, where n and m are the lengths of `num1` and `num2` respectively.

Space Complexity: $O(n + m)$, used for storing the intermediate multiplication results.

Solution

The solution multiplies each digit of `num1` with each digit of `num2` by simulating the grade-school multiplication algorithm. An array is used to store the intermediate sums at positions that correspond to the sum of the indices from `num1` and `num2`. For two digits multiplied together, the product is added to the corresponding index in the result array, and any carry is added to the adjacent left cell. Finally, the array is converted back to a string while taking care to remove any leading zeros.

Data structures used:

- Array for intermediate multiplication results.
- Two nested loops to multiply each digit pair.

Algorithmic steps:

- Initialize an array of size $(\text{length of num1} + \text{length of num2})$ with all zeros.
- Loop through digits of `num1` and `num2` in reverse order.
- Multiply the two digits, add

p.241

the product to the corresponding position in the result array (taking into account any previous values).

- Manage the carry for each index.
- Convert the result array to a string, skipping any leading zeros.
- Return the final product string.

p.242

#2013 Detect Squares

MED

Key Insights

- Leverage the fact that an axis-aligned square has vertical and horizontal edges.
- For a query point (x, y) , iterate over all points that share the same y -axis. For each of these points (px, y) (with $px \neq x$), determine the potential square's side length, $d = |px - x|$.
- Check for the existence of the two other points $(x, y+d)$ and $(px, y+d)$ (or $(x, y-d)$ and $(px, y-d)$) that would complete the square.
- Use a hash map (or dictionary) to store each point along with its count since duplicate points are allowed.

Space & Time

Time Complexity:

- The add operation is $O(1)$.
- The count operation is $O(n)$ in the worst-case scenario, where n is the number of distinct points with the same y -coordinate as the query point.

Space Complexity:

- $O(m)$, where m is the number of added points, since we store counts for each point.

Solution

We implement the `DetectSquares` class by maintaining a hash map (or dictionary) to store the frequency count for each point using its (x, y) coordinates as the key. In the `add(point)` method, we simply increment the count for the given point. For the `count(point)` method, we iterate through all points that share the same y -coordinate as the query point. For every such point (px, y) (with $px \neq x$), we calculate the potential side length of a square ($d = px - x$). Then, we check for two possible squares: one above the query point and one below it. The count is increased by the number of times these needed points exist multiplied by the count

p.243

of the current candidate point. This method efficiently counts all possible squares that can be formed with the query point.

p.244

#371 Sum of Two Integers

MED

Key Insights

- Use bitwise XOR ($\wedge$) to perform addition without carrying.
- Use bitwise AND ($\&$) followed by a left shift ($\ll$) to calculate the carry.
- Repeat the process until there is no carry.
- This method simulates the addition process using binary arithmetic.

Space & Time

Time Complexity: $O(1)$ – The loop runs a fixed number of times based on the number of bits.

Space Complexity: $O(1)$ – Only constant extra space is used.

Solution

The solution uses bit manipulation to add two numbers without using the $+$ or $-$ operators.

- Use XOR ($\wedge$) on a and b to add the bits where there is no carry.
- Use AND ($\&$) to find the positions where both a and b have a 1 (i.e., potential carry). Then shift the result left by one to add in the next higher bit.
- Set a to the XOR result and b to the shifted carry.
- Repeat steps 1–3 until there is no carry (b equals 0).
- Return a which holds the final sum. This approach leverages the bitwise operations to effectively simulate binary addition.

p.245